物理を組み込む機械学習

講演の解説 —— 第 34 回鉄鋼工学アドバンストセミナー 基調講演 (2026-10-19)

村田 昇

目次

要旨 2

1 はじめに: フィジカル AI と物理の知識 2
- 1.1 フィジカル AI とは . . . 2
- 1.2 現実世界の壁 . . . 2
- 1.3 PIML の役割と本稿の構成 . . . 3

2 物理の組み込み方: 3 つのバイアス 3
- 2.1 物理を組み込む 3 つの場所 . . . 4
- 2.2 学習の枠組みと 3 つのバイアス . . . 4
- 2.3 正則化の系譜 . . . 5
- 2.4 支配方程式と境界条件 . . . 6

3 バイアスを実現する技術 6
- 3.1 観測バイアスの実現: データに組み込む . . . 7
- 3.2 学習バイアスの実現: 方程式のずれを罰則に . . . 7
- 3.3 自動微分: 学習した関数の導関数 . . . 8
- 3.4 数値積分: 方程式もデータと同じ形 . . . 8
- 3.5 帰納バイアスの実現 1: 出力の形で満たす . . . 9
- 3.6 帰納バイアスの実現 2: 積分層と射影層 . . . 9
- 3.7 帰納バイアスの実現 3: ハミルトン力学 . . . 11
- 3.8 関数の学習と作用素の学習 . . . 12
- 3.9 関数近似: 関数と作用素の万能近似 . . . 13
- 3.10 PINN と PINO の統一的な定式化 . . . 13
- 3.11 何を求めるか: 順問題・設計問題・逆問題 . . . 14

4 事例研究: 制約付き曲線の設計 15
- 4.1 題材と共通の記法 . . . 15
- 4.2 問題 1: 順問題 . . . 17
- 4.3 問題 2: 設計 . . . 18
- 4.4 問題 3: 同定 . . . 19
- 4.5 問題 4: 曲率分布の設計 . . . 21
- 4.6 問題 5: 条件を変えて何度も解く . . . 22
- 4.7 問題 6: 作用素学習 . . . 23
- 4.8 速度と幾何: 走れる軌道 (問題 7a・7b) . . . 25
- 4.9 事例のまとめ . . . 26

5 古典的な数値解析との使い分け 27

6 課題と展望 28

7 研究の歩み 29
- 7.1 技術の年表 . . . 29

8 むすび 29

9 生成 AI の利用について 30

10 付録: 数値実験の設定 30

参考文献 30

要旨

現実の物理空間で知覚・判断・行動する AI (フィジカル AI) では，データの乏しさ，物理的に不可能な予測，シミュレーションと現実の乖離，実時間性という固有の制約が問題になる．物理の知識を学習に組み込む機械学習 (physics-informed machine learning，PIML) は，これらに対する有力な手がかりである．本稿ではまず，PIML を「データの損失＋正則化＋仮説空間」という古典的な学習の枠組みの素直な拡張として見直し，物理を組み込む 3 つの場所 (観測・学習・帰納バイアス) がこの枠組みの 3 つの部分にそのまま対応すること，罰則による学習バイアスと構造による帰納バイアスの対比が Tikhonov 正則化と Ivanov 正則化の対比に重なることを示す．次に，3 つのバイアスを実現する技術を，物理を反映したデータ，物理残差の罰則とそれを支える自動微分・数値積分，学習器の後ろに固定の写像を置く帰納バイアス (試行解，微分可能な積分層と射影層，ハミルトン力学の構造) の順に説明し，関数から作用素への学習の拡張と万能近似定理を経て，PINN と PINO を条件 a についての平均を持つ一つの損失として定式化する．後半では，道路や車両の経路設計に現れるクロソイド曲線を題材に，順問題・設計・同定から，曲率分布そのものを求める問題，条件を変えて繰り返し解く問題，関数を入力とする作用素学習，動特性を含む軌道生成までの 8 つの問題を，学習器・学習する量・方程式・境界・データの損失という同一の表で定式化し，数値実験の結果とその意味を示す．最後に古典的な数値解析との使い分けを整理し，「確実に計算できる部分は古典解法に，関数の未知と繰り返しは学習に任せる」という設計原理と残された課題を議論して，技術の年表とともに研究の歩みを振り返る．

1 はじめに: フィジカル AI と物理の知識

1.1 フィジカル AI とは

フィジカル AI (physical artificial intelligence) は，デジタル空間での計算や情報処理にとどまらず，現実の物理空間で自律的に知覚し，判断し，行動する AI システムとその技術領域を指す．従来の AI が文章・画像・音声などのデジタルデータの処理に特化していたのに対し，フィジカル AI はロボット，自動運転車，ドローンなどを制御し，身体 (embodiment) を持って現実の世界で動く．Miriyev & Kovač (2020) はこれを「材料科学，機械工学，計算機科学，生物学などの技能を融合させ，生命のような知能を持つロボットを合成するための理論と実践」と位置付け，学術的な概念として提唱した．工学的な体系化は Thakur et al. (2025) が整理しており，産業界でも学習・シミュレーション・実行の 3 つの計算基盤という形で急速に普及している．

フィジカル AI の本質は，「見る (センサー) → 判断する (AI) → 動く (アクチュエータ)」の閉ループを現実の時間の中で回し続けることにある．動けば環境が変わり，それをまた観測して判断し直す．この点が，入力から出力への一回きりの写像を学ぶ画面の中の AI との根本的な違いである．目指すものは，(1) 散らかった部屋のように物の配置が毎回変わる非構造化環境で動くこと，(2) 言葉による指示を理解し人のそばで安全に作業すること，(3) 予期しない出来事に実時間で応じること，の 3 つに要約できる．いずれも，前もってすべてを決めておけず，現実の状況に合わせてその場で動作を生成しなければならない点で共通する．

これを支える技術は，大きく 4 つに整理できる．第一は物理を模した仮想空間で大量の試行を行い，獲得した方策を実機に移す技術 (シミュレーション，デジタルツインと Sim-to-Real) である．第二は映像と言語から行動指令を出力し，物の動きを予測する大規模な基盤モデル (vision-language-action，VLA モデルや世界モデル) である．第三は実機の動作データを集め，危ない動きをしないことを確かめる仕組み (データの収集と安全の検証) である．第四は，力や触覚を捉えてその場で素早く制御する身体 (アクチュエータ，センサ，機体に積む計算機) である．次節の 4 つの壁はこの順に対応する．物理の知識はどれにも関わる: 仮想空間の忠実度，予測の物理的妥当性，少ないデータからの学習，速い模型である．

1.2 現実世界の壁

フィジカル AI を現実の世界で安全かつ実用的に使うには，次の 4 つの壁がある．

- **ずれ (仮想と現実，Sim-to-Real ギャップ)**: 摩擦，空気抵抗，センサーの雑音，通信遅延など，模型に十分入っていない物理的要因の違いで，仮想空間で学んだ方策の性能が実機で大きく落ちる．シミュレーションで覚えた歩き方や物の持ち方が実機では転ぶ・落とす，路面の摩擦やタイヤの特性，機体の質量といった小さな違いが積み重なって走行がずれる，といった形で現れる．
- **データ (現実のデータは高くつく)**: 物理世界のデータ (ロボットの動作，車両の走行，流れの計測など) は収集に時間がかかる．実機の走行試験や道路の測量には時間と費用がかかり，転倒や衝突を伴う試行錯誤は機体を壊す．インターネット規模のデータで学習する言語・画像のモデルとは事情が根本的に異なる．
- **安全 (法則を破る予測は危ない)**: データだけから学んだモデルは，質量やエネルギーの保存などの法則に反する予測を出すことがある．物体が床をすり抜ける，曲がりきれない急カーブを指示する，といった出力は，現実の環境では事故に直結しうる．珍しい状況ほどデータが少なく，予測が外れやすい．
- **速さ (一瞬で判断しなければならない)**: 制御の周期は数〜数十 ms で，状況の変化にはミリ秒単位の応答が求められる．一方，機体に積める計算機は小さく，電力も限られる．大規模なモデルや重い物理シミュレーションをそのまま動かすことは難しい．

1.3 PIML の役割と本稿の構成

これらの壁に対する有力な手がかりが，物理法則を事前知識として機械学習のモデルに組み込む PIML (physics-informed machine learning) である (Karniadakis et al., 2021)．physics-informed は「物理法則を組み込んだ」の意味で，本稿では短く「物理を組み込んだ」と書く．物理の知識は，(1) 観測の少ない所を法則が補うことで必要なデータを減らし，(2) 出力を法則に沿わせることで物理的にありえない予測を抑え，(3) 観測から未知の係数を同定して模型を現実に合わせ，データの外でも法則に基づくもっともらしい推論を与えることで Sim-to-Real の頑健性を高め，(4) 重い数値シミュレーションを学習済みの代理モデル (サロゲート) で置き換えることで実時間の予測を可能にする．4 つの壁と PIML の答え，それを確かめる本稿の事例 (4 節) の対応を表にまとめる．

表 1: 現実世界の壁と PIML による答え

壁	PIML による答え	本稿の事例
データ: 高くつく	少ないデータでも学べる (観測の隙間を法則が補う)	問題 3: 5 点の測量から全区間を再現
安全: 法則を破る予測	ありえない動きを出しにくい (出力を法則に沿わせる)	問題 4・7: 終点・最小半径・走行の限界を守る
ずれ: 仮想と現実	未知の状況でも，もっともらしく予測 (係数の同定)	問題 3: 観測から曲線の係数を同定
速さ: 一瞬の判断	学習済みの模型で，速く予測 (代理モデル)	問題 5: 1 件 0.7 ms の推論

本稿の構成は次のとおりである．2 節では，PIML を「データの損失＋正則化＋仮説空間」という古典的な学習の枠組みの素直な拡張として見直し，物理を組み込む 3 つの場所 (観測・学習・帰納バイアス) と，正則化の考え方の系譜を整理する．3 節では，3 つのバイアスを実現する具体的な技術 (物理を反映したデータ，物理残差の罰則，試行解，積分層と射影層，ハミルトン力学の構造) と，それを支える自動微分・数値積分・関数と作用素の万能近似を説明し，PINN と PINO を一つの損失で定式化する．4 節ではクロソイド曲線の設計を題材とした 8 つの問題を，統一した表で定式化して数値実験の結果を示す．5 節で古典的な数値解析との関係を整理し，6 節で残された課題を議論する．7 節では，技術の年表を示す．PINN の包括的なサーベイとしては Cuomo et al. (2022)，工学・環境分野での科学知識と機械学習の統合については Willard et al. (2022)，作用素学習の体系的な解説としては Kovachki et al. (2023) を併せて参照されたい．

2 物理の組み込み方: 3 つのバイアス

物理の知識を学習に組み込む方法は多様に見えるが，入れる先は 3 つしかない．本節では，その 3 つ (観測・学習・帰納バイアス) が，データの損失と正則化と仮説空間からなる古典的な学習の枠組みのそれぞれの部分にそのまま対応することを示し (2.1〜2.2 節)，正則化の考え方の系譜の中に PIML を位置づける (2.3 節)．最後に，以下で用いる支配方程式と境界条件の記号を定める (2.4 節)．

2.1 物理を組み込む 3 つの場所

Karniadakis et al. (2021) は，物理の知識を機械学習に組み込む方法を，観測バイアス，帰納バイアス，学習バイアスの 3 つに分類した．ここでのバイアスは日常語の「偏見」ではなく，データだけでは決まらない部分を補うために学習の側があらかじめ持つ「好ましい偏り」，すなわち先入観や事前の仮定を指す．数学的には，解の候補 (探索空間) を絞り込む制約として働く．機械学習では，学習器が観測していない入力に対して汎化するために置く仮定を総称して帰納バイアス (inductive bias) と呼んできた (Mitchell, 1997)．PIML の 3 つのバイアスは，その仮定として物理法則を使い，どこに置くかで分けたものと言える．

- **観測バイアス**: 物理を **データ**に組み込む．実験や観測，数値シミュレーションの結果，対称性を使ったデータ拡張など，物理を体現したデータを学習に与える．問うのは「何を見せるか」である．法則が守られるのはデータが覆う範囲で，近似的にだけである．
- **学習バイアス**: 物理を **学習の目標**に組み込む．方程式のずれを罰則として損失に加えるなど，損失関数や最適化の仕方を工夫して，学習が物理法則に従う解へ向かうように誘導する．問うのは「何を良しとするか」である．法則は罰則の重みと学習の進み方に応じて満たされる (ソフト制約)．
- **帰納バイアス**: 物理を **学習器の形**に組み込む．学習器の構造を工夫して，どんなパラメータでも出力が物理法則を満たすようにする．問うのは「何を表せるか」である．法則は構造上つねに満たされる (ハード制約)．

表 2: 物理を組み込む 3 つの場所

バイアス	物理を組み込む先	問い	代表例	満たし方
観測バイアス	データ	何を見せるか	シミュレーションの代理モデル，対称性によるデータ拡張	データの範囲で近似的に
学習バイアス	学習の目標	何を良しとするか	PINN，PINO	罰則の重みに応じて (ソフト制約)
帰納バイアス	学習器の形	何を表せるか	境界条件を満たす試行解，積分層と射影層，HNN，同変 NN	構造上つねに (ハード制約)

帰納バイアスと学習バイアスは，どちらも解の候補を物理を満たす方へ寄せるが，前者は候補の外に出られないようにし，後者は外に出ると損をするようにする．観測バイアスは最も手軽だが外挿に弱い．帰納バイアスは満たすべき性質を構造で保証するので強力だが，構造の設計が問題ごとに必要になる．学習バイアスは方程式さえ書ければ汎用的に使えるが，満たし方は近似的で重みの選択に依存する．実際の手法はこれらを組み合わせることが多く，4 節の事例でも，前半 (問題 1〜3) は学習バイアス (PINN) を中心に，後半 (問題 4 以降) は運動学と終点条件を帰納バイアス (積分層と射影層) に移していく．この「ソフトからハードへ」の移行は本稿の一つの軸である．なお，データから支配方程式そのものを発見する方向 (疎回帰による SINDy (Brunton et al., 2016) など) も PIML の重要な一分野だが，本稿では扱わない．

2.2 学習の枠組みと 3 つのバイアス

機械学習の多くの手法は，学習器 f を仮説空間 $\mathcal{F}$ (学習器で表せる関数の集合) の中から選び，データへの当てはまり (観測の損失) に正則化項を足した量を最小化する．

$$\min_{f\in\mathcal{F}} \frac{1}{n}\sum_{k=1}^{n} \ell(f(x_k), y_k) + \lambda\,\mathcal{R}[f].$$

(x_k, y_k) は n 組の観測，ℓ は損失 (たとえば 2 乗誤差 $\ell(u,y)=|u-y|^2$)，$\mathcal{R}$ は正則化項，$\lambda>0$ は正則化の強さである．学習器がパラメータ $\theta\in\Theta$ を持つ関数 f_θ (ニューラルネットワーク (NN) など．NN では θ は重みとバイアス) なら $\mathcal{F}=\{f_\theta : \theta\in\Theta\}$ で，f についての最小化は θ についての最小化になる．正則化項は「解は滑らかなはず」「係数は小さいはず」といった事前知識を表し，データが少ない，あるいは雑音を含むときに解を安定させる．

物理を組み込む 3 つの場所は，この式の 3 つの部分にそのまま対応する (図)．データ項に物理を組み込むのが観測バイアス，正則化項 $\mathcal{R}$ に物理を組み込む (方程式のずれを罰則にする) のが学習バイアス，仮説空間 $\mathcal{F}$ に物理を組み込む (物理を満たす関数しか表せない学習器を使う) のが帰納バイアスである．

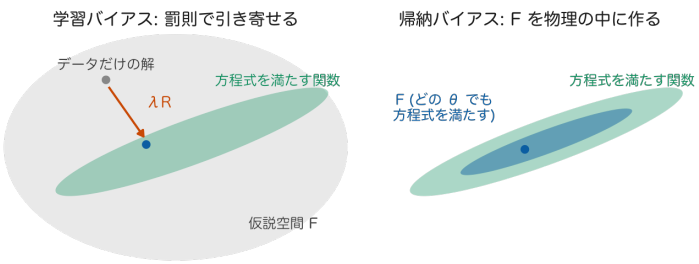


図 1: 学習バイアスと帰納バイアスの違い．灰色は仮説空間 $\mathcal{F}$，緑は方程式を満たす関数の集合．左: 罰則 $\lambda\mathcal{R}$ でデータだけの解を引き寄せる．右: $\mathcal{F}$ 自体を緑の内側に作る

学習バイアスでは，仮説空間 $\mathcal{F}$ は広いままで，データだけで選ぶと方程式を満たさない解 (図左の灰色の点) になりうる．支配方程式を $\mathcal{N}[u] = 0$ (2.4 節) として物理の残差 $\mathcal{R}[f] = \|\mathcal{N}[f]\|^2$ を正則化項に使えば，罰則 $\lambda\mathcal{R}$ が解を方程式を満たす関数の集合の方へ引き寄せ，観測のない領域でも解を決める．λ が小さいと引き寄せが弱く，大きいとデータへの当てはまりが犠牲になる．データ項に重み w_d を付けて書けば w_d は $1/\lambda$ にあたり，古典的な正則化で λ の選択 (L 曲線法，交差検証など) が重要であるのと同じく，その選び方が結果を大きく左右する (4.4 節)．帰納バイアスでは，学習器の形を工夫して $\mathcal{F}$ 自体を方程式を満たす関数の集合の内側に作る (図右)．どの θ を選んでも方程式は満たされ，学習は満たす関数の中で「どれがよいか」だけを決める．観測バイアスは，損失の和をとるデータの集合 $\{(x_k, y_k)\}$ に，物理を反映したデータを加えることにあたる．

この枠組みはベイズ的にも読める．雑音を正規分布とすると，2 乗誤差のデータ項は尤度の負の対数，正則化項 $\lambda\mathcal{R}[f]$ は事前分布の負の対数にあたり，損失の最小化は事後確率の最大化 (MAP 推定) になる．物理の罰則 $\lambda\|\mathcal{N}[f]\|^2$ は「方程式を満たす関数ほど起こりやすい」という事前分布 $p(f) \propto \exp(-\lambda\|\mathcal{N}[f]\|^2)$，帰納バイアスは「方程式を満たさない関数は確率 0」という，台が方程式の解の集合に限られた事前分布と読める．観測バイアスは尤度の側，すなわちどの観測を条件にするかの選択である．

要するに，物理を使った学習は古典的な学習の枠組みを変えるものではなく，その素直な拡張である．新しいのは，正則化や仮説空間に方程式 (導関数や積分を含む条件) を入れても計算できるようにした技術の方である．3 節で，それを 3 つのバイアスごとに見る．

記号: 本稿では，最終的に求めたい関数 (方程式の解) を u_θ，学習するパラメトリックな関数 (NN など) を学習器 f_θ，正則化項を $\mathcal{R}$，仮説空間を $\mathcal{F}$ と書く．学習バイアスだけを使うときは $u_\theta = f_\theta$ である．帰納バイアスを使うときは，u_θ は f_θ に学習しない固定の写像 Φ を施したもの $u_\theta = \Phi(f_\theta)$ になる (3.5〜3.7 節)．

2.3 正則化の系譜

関数そのものに条件を課して解を安定させる考え方は，PIML よりはるかに古い．観測の作用素を A とする逆問題 $Af = y$ (学習では A は点 x_k での値を取り出す作用素) に対して，正則化には 2 つの書き方がある．

$$\text{Tikhonov:} \quad \min_f \ \|Af - y\|^2 + \lambda\|Lf\|^2,$$

$$\text{Ivanov:} \quad \min_f \ \|Af - y\|^2 \quad \text{subject to} \quad \|Lf\| \le \tau.$$

Tikhonov の正則化は正則化項 $\lambda\|Lf\|^2$ を罰則として損失に足し (ソフト制約)，Ivanov の正則化は解の候補を $\|Lf\| \le \tau$ の集合に制限する (ハード制約)．L には恒等作用素 (解の大きさを抑える) や微分作用素 (滑らかさを要求する) が選ばれる．凸な問題では，ラグランジュ乗数を通じて λ と τ が単調に対応し，両者は同じ解の族を与える．すなわち，同じ問題の 2 つの書き方である．両者の関係を含む古典的な理論は教科書 (Tikhonov & Arsenin, 1974/1977) にまとめられている．

この対比は，学習バイアスと帰納バイアスの対比にそのまま重なる．PINN のように方程式のずれを罰則にするのは Tikhonov 型である．一方，Ivanov の書き方で L を支配方程式の作用素 $\mathcal{N}$，$\tau = 0$ とすると，候補の集合は「方程式をちょうど満たす関数の集合」になり，帰納バイアスの仮説空間と一致する．これが帰納バイアスの原型である．違いは制約の守らせ方にあり，Ivanov は最適化の側で制約付き問題として守らせ，帰納バイアスは学習器の形 (パラメータの取り方) によって最初から集合の外に出られないようにする．後者では θ について制約のない問題になるので，ふつうの勾配法がそのまま使える．なお，罰則の重みを大きくしていけば Tikhonov 型の解は制約を満たす解に近づくが，有限の重みでは制約はわずかに破れ，重みを大きくするほど問題の条件が悪くなる (最適化の罰則法と同じ事情)．厳密な充足が要る所で帰納バイアスが効くのはこのためである (4.5 節の問題 4)．

正則化項に何を使うかには，おおよそ次の移り変わりがある．

1. **パラメータの大きさ**: 一般によく知られている正則化は，重み減衰 $\|\theta\|^2$ のようなパラメータの大きさへの罰則である．学習器がパラメトリックな関数なら，パラメータへの制約は暗黙のうちに関数空間への制約になっているが，それがどんな関数を好むかは学習器の構造を通してしか分からない．
2. **関数の微分**: 関数そのものの滑らかさに条件を課す考えも古い．平滑化スプラインは $\sum_k (y_k - f(x_k))^2 + \lambda \int f''(x)^2 dx$ を最小化し，解が自然 3 次スプラインになる．この考え方は再生核ヒルベルト空間の正則化理論として体系化された (Wahba, 1990)．NN では，Moody (1991) が重みの大きさに限らない一般の正則化項を扱い，正則化によって模型の実効的な自由度 (有効パラメータ数) が名目上のパラメータ数より小さくなることを示して，汎化誤差の見積もりに用いた．その後も，学習器の出力の入力についての 2 階微分 (曲率) を罰する平滑化や，滑らかさの汎関数と NN の構造を結びつける正則化理論が続く (Bishop, 1995)．微分を使う正則化は，ソボレフ空間のような関数空間で解を探すことにあたり，NN の近似理論 (Barron 空間など．3.9 節) ともつながる．
3. **方程式**: PINN の物理残差 $\|\mathcal{N}[u_\theta]\|^2$ は，この正則化項を支配方程式のずれに置き換えたものである．考え方そのものは古いが，学習器の高階導関数を入れ子で計算して重みの勾配まで求めるのは手作業では難しく，自動微分の普及 (3.3 節) で初めて実用になった．Lagaris et al. (1998) の時点では，導関数の式を手で導いて実装していた．

罰則のノルムの選び方も解の性質を決める．4.5 節の問題 4 で使う滑らかさの項 $\int |\kappa''|\, ds$ は「2 階微分の L1 罰則」で，時系列の ℓ_1 トレンドフィルタ $\min_x \frac{1}{2}\sum_t (y_t - x_t)^2 + \lambda \sum_t |x_{t-1} - 2x_t + x_{t+1}|$ (Kim et al., 2009) と同じ形である．L1 は疎な解を好むので 2 階差分がほとんど 0 になり，解は区分線形になる．曲率 κ が区分線形とは，直線・クロソイド・円弧の組合せにほかならない．L2 の平滑化スプラインとの違いは，折れ点を保つか角を丸めるかである．

2.4 支配方程式と境界条件

以下では, 解 u が領域 Ω の中で満たす支配方程式を $\mathcal{N}[u;\lambda] = 0$, 領域の境界や初期時刻で満たす条件を $\mathcal{B}[u] = g$ と書く．$\mathcal{N}$ は u とその導関数を含む (一般に非線形の) 微分作用素で，λ は方程式の係数 (物性値など)，$\mathcal{B}$ は境界での値や導関数を取り出す作用素 (ディリクレ条件，ノイマン条件など)，g はその目標値である．記号 $\mathcal{N}$ は Raissi らの原論文以来 PINN の文献で広く使われているものに従った．

表 3: 支配方程式と境界・初期条件の例

例	支配方程式 $\mathcal{N}[u;\lambda] = 0$	境界・初期条件 $\mathcal{B}[u] = g$	物理パラメータ λ
熱方程式	$\partial_t u - \alpha \partial_x^2 u = 0$	$u(x,0) = u_0(x)$，$u(0,t) = u(1,t) = 0$	熱拡散率 α
ばね・質量	$m\ddot{u} + c\dot{u} + ku = F(t)$	$u(0) = u_0$，$\dot{u}(0) = v_0$	m, c, k
曲線 (4 節)	$x' - \cos\varphi = 0$，$y' - \sin\varphi = 0$，$\varphi' - \kappa = 0$	$(x,y,\varphi)(0) = (0,0,0)$，終点の条件	曲率 $\kappa(s)$

どの例も「導関数を含む方程式」と「端での条件」の組であり，条件がなければ解は一つに定まらない．PIML では，この方程式と条件のずれを損失に入れる (学習バイアス) か，満たす関数だけを作る (帰納バイアス).

3 バイアスを実現する技術

2 節の枠組みで物理を組み込む場所は決まった．それを実際に計算するには技術が要る．本節では，3 つのバイアスを実現する具体的な方法を順に説明し (観測: 3.1 節，学習: 3.2 節，帰納: 3.5〜3.7 節)，学習バイアスを支える自動微分と数値積分 (3.3〜3.4 節) をその間に置く．続いて，学ぶ対象を関数から作用素に広げ (3.8 節)，その表現の根拠である万能近似定理を確認し (3.9 節)，PINN と PINO を一つの損失で定式化する (3.10〜3.11 節)．PINN や PINO は単一の発明ではなく，機械学習・深層学習・応用数学の複数の要素技術の発展が合流して成立したものである．

表 4: 3 つのバイアスを実現する技術

バイアス	実現の仕方	支える技術	本稿の事例
観測バイアス	物理を反映したデータ，対称性によるデータ拡張	シミュレーション	問題 3 の測量点

バイアス	実現の仕方	支える技術	本稿の事例
学習バイアス	物理残差を正則化項にする (PINN，PINO)	自動微分，数値積分	問題 1〜3，問題 5・7b の罰則
帰納バイアス	学習器の後ろに固定の写像 Φ を置く: 試行解，積分層と射影層，ハミルトン力学	自動微分，微分可能な数値計算	問題 1〜3 の始点，問題 4〜7b
(共通)	解や「条件 $\to$ 解」の写像を学習器で表す	関数・作用素の万能近似	MLP，条件付き PINN，FNO

3.1 観測バイアスの実現: データに組み込む

観測バイアスは，物理を反映したデータを学習に与える方法である．学習器も損失も変えずに使えるので，最も手軽である．データの出どころには次のものがある．

- **実験・観測のデータ**: 現実の物理をそのまま反映する．ただし高価で，量が限られる (1.2 節)．
- **シミュレーションで作ったデータ**: 数値シミュレーションの入出力の組を大量に作り，高価なシミュレーションの代わりになる速い模型 (代理モデル) を学ぶ．データに含まれる物理は，シミュレーションの模型の忠実度で決まる．
- **対称性によるデータ拡張**: 物理法則が変換の群 G (回転，平行移動，反転など) で変わらないなら，観測 (x, y) を変換した $(g \cdot x,\, g \cdot y)$ $(g \in G)$ もやはり正しいデータである．これを学習データに加える．

曲線の例 (4.1 節) では，曲線を x 軸について反転したもの，すなわち (x, y, φ, κ) を $(x, -y, -\varphi, -\kappa)$ に置き換えたものも同じ方程式を満たすので，データ拡張に使える．始点の位置や向きを変える平行移動・回転も同様である．同じ対称性を学習器の構造で保証すれば，同変 NN (3.7 節) という帰納バイアスになる．

観測バイアスの限界は，法則が守られるのがデータの覆う範囲で，近似的にだけという点である．データのない所 (外挿) では，物理的にありえない予測を出しうる．4.4 節の問題 3 では，5 点の測量だけで学んだ NN は測量点を通るものの，未測量の区間では真の曲線から 2.9〜3.7 m 外れた．物理残差 (学習バイアス) を加えると 0.17〜1.7 m (5 試行中 3 試行は 0.25 m 以下) に下がる．

3.2 学習バイアスの実現: 方程式のずれを罰則に

学習バイアスの基本形は，支配方程式 $\mathcal{N}[u] = 0$ のずれ (物理残差) を正則化項にすることである．

$$\mathcal{R}[u_\theta] = \int_\Omega \left|\mathcal{N}[u_\theta](x)\right|^2 dx \;\approx\; \frac{1}{n_r}\sum_{i=1}^{n_r} \left|\mathcal{N}[u_\theta](x_i)\right|^2.$$

本来は領域 Ω 上の積分だが，コロケーション点 x_i での和で近似する．境界条件 $\mathcal{B}[u] = g$ のずれも同様に境界上の点での和で罰則にする．学習器 $u_\theta = f_\theta$ の形は変えず，損失に項を足すだけなので，方程式さえ書ければどんな問題にも使える．これが PINN (Raissi et al., 2019) の基本形である．

この式を計算するには 2 つの技術が要る．残差 $\mathcal{N}[u_\theta]$ には導関数が入るので，学習器を入力 x について微分する必要がある (自動微分，3.3 節)．積分は点での和にすれば，データの損失と同じ「点の上の誤差の平均」の形になる (数値積分，3.4 節)．どちらも微分可能な計算なので，重み θ についての勾配が機械的に計算でき，確率的勾配法でそのまま学習できる．

曲線の例では，弧長を区間長 L で無次元化した $t = s/L \in [0, 1]$ を入力とし，学習器が解そのものを表す $(f_\theta : t \mapsto (x, y, \varphi)$，$u_\theta = f_\theta)$．$' = d/dt$ とすると $ds = L\, dt$ なので，方程式の残差と正則化項は

$$r(t) = (x' - L\cos\varphi,\; y' - L\sin\varphi,\; \varphi' - L\kappa),$$
$$\mathcal{R} = \frac{1}{n_r}\sum_{i=1}^{n_r} \left|r(t_i)\right|^2$$

となる (4.1 節の $\mathcal{L}_r$)．方程式は損失が下がった分だけ近似的に満たされ，罰則の重みのつり合いに結果が左右される (問題 1〜3)．2.3 節で述べたとおり，重みを大きくすれば方程式はよりよく満たされるが，問題の条件が悪くなり学習が難しくなる．

損失の形には変種がある．強形式の残差の代わりにエネルギー汎関数を最小化する Deep Ritz 法 (E & Yu, 2018)，弱形式 (テスト関数との内積) を使う VPINN (Kharazmi et al., 2019)，ランダムに選んだ点での残差を確率的勾配法で減らす DGM (Sirignano & Spiliopoulos, 2018) などである．弱形式は要求する導関数の階数を下げ，求積を正確にできる利点がある．実装上の基盤としては DeepXDE (Lu, Meng, et al., 2021) などのライブラリが整備されている．

3.3 自動微分: 学習した関数の導関数

物理法則の多くは微分方程式で書けるので，PINN の損失を評価するには学習中の関数 u_θ の導関数が必要になる．NN は線形変換と活性化関数の合成なので，連鎖律を機械的に適用すれば入力 x についての導関数が得られる．これを計算機で自動的に行うのが自動微分 (automatic differentiation，AD) である (Baydin et al., 2018)．

数値微分 (差分) は実装が簡単だが，刻み幅を小さくすると丸め誤差 (桁落ち) が，大きくすると打切り誤差が増える．さらに，入力次元に比例する回数の関数評価が要る．記号微分 (数式処理) は厳密だが，合成を重ねるごとに式が膨張しうる．自動微分は計算を基本演算の計算グラフとみなし，各演算の局所的な導関数を連鎖律で組み合わせる．そのため丸め誤差を除いて厳密で，コストは元の計算の定数倍に収まる．伝え方には 2 通りある．前進モードは入力側から 1 方向ずつ導関数を伝え，入力が少なく出力が多いときに有利である．後退モードは出力側から逆向きに伝え，スカラーの出力 (損失) に対する全パラメータの勾配を一度に得る．誤差逆伝播法は後退モードの特別な場合である．

PINN では，重み θ に関する勾配 (学習用) に加えて，入力座標 x に関する導関数 (残差の評価用) が必要になる．PyTorch であれば `torch.autograd.grad` に `create_graph=True` を指定すると導関数の計算自体が計算グラフに残り，それをさらに微分して 2 階導関数を得たうえで，最後に損失を θ について微分できる．すなわち「入力についての微分を，重みについてもう一度微分する」入れ子の自動微分である．PyTorch，TensorFlow，JAX でこれが数行で書けるようになったことが，PINN の成立の最大の実装上の鍵であった．

高階微分のコスト: k 階の導関数を入れ子の自動微分で計算すると，計算グラフが段ごとに大きくなり，計算時間とメモリが増える．d 次元のラプラシアンは素朴には d 回の逆伝播を要し，学習ではさらにそれを θ について微分する．対策として，テイラー級数の係数をまとめて伝えるテイラーモードの自動微分や，ランダムな方向の微分による推定，格子上の関数なら FFT による微分 (4.7 節の問題 6) がある．

活性化関数の滑らかさ: 残差に k 階の導関数が入るなら，u_θ は少なくとも C^k でなければならない．ReLU は区分線形なので 2 階導関数がほとんど至るところ 0 になり，2 階の微分方程式の残差を評価できない．このため PINN では tanh が標準的に使われてきた．GELU や SiLU ($x\,\sigma(x)$) は滑らかで深い NN でも勾配が流れやすく，正弦関数を活性化に使う SIREN (Sitzmann et al., 2020) は任意階の導関数が同じ形になるので導関数を含む損失と相性がよい．残差結合や層正規化は深い NN の学習を安定させる．

積分を計算に組み込む: 導関数だけでなく，積分も微分可能な演算で書けば学習に組み込める．台形則の累積和のように，積分を和で書けば，その結果に対する損失の勾配も自動微分で θ まで伝わる．これは帰納バイアスの一つである積分層 (3.6 節) の基礎になる．

3.4 数値積分: 方程式もデータと同じ形

方程式と境界条件の損失は本来，領域とその境界の上の残差の積分 (ノルム) であり，データの損失は本来，観測の分布についての期待値である．実際の計算では，どちらも点での和で近似する．

$$
\begin{aligned}
\mathcal{L}_r &= \int_\Omega \left|\mathcal{N}[u_\theta](x)\right|^2 dx \;\approx\; \frac{1}{n_r}\sum_{i=1}^{n_r} \left|\mathcal{N}[u_\theta](x_i) - 0\right|^2, \\
\mathcal{L}_b &= \int_{\partial\Omega} \left|\mathcal{B}[u_\theta](x) - g(x)\right|^2 dx \;\approx\; \frac{1}{n_b}\sum_{j=1}^{n_b} \left|\mathcal{B}[u_\theta](x_j) - g_j\right|^2, \\
\mathcal{L}_d &= \mathbb{E}\left|u_\theta(x) - y\right|^2 \;\approx\; \frac{1}{n_d}\sum_{k=1}^{n_d} \left|u_\theta(x_k) - y_k\right|^2.
\end{aligned}
$$

両者は「点の上の誤差の 2 乗平均」という同じ形になり，方程式の項は「各点で残差が 0 という観測」を持つ仮想のデータとみなせる．これにより，方程式・境界・観測の 3 項を，ミニバッチや確率的勾配法といった同じ学習の枠組みでまとめて扱える．この見方は，PIML を統計的学習の言葉で理解するうえで基本的である．

コロケーション点の取り方は，データの標本の取り方にあたる．一様乱数，格子，準乱数のほか，残差の大きい所に点を増やす適応的サンプリングがある．有限個の点での和は積分の近似なので，点の取り方が偏れば損失も偏る．特に，積分が損失の中や観測モデルの中に入れ子になるとき，有限和の誤差が学習の偏り (biased loss) を生む．

同じ構造は，3 次元シーンを座標の多層 NN (MLP) で表す NeRF (Mildenhall et al., 2021) にも見られる．NeRF は光線に沿った密度と色の積分を光線上の点の有限和で近似し，撮影画像との差を最小化する．積分が観測モデル (順問題) の中にあるか損失の定義の中にあるかの違いはあるが，「座標を入力とする NN ＋有限和の求積」という構造を共有し，Fourier 特徴量 (Tancik et al., 2020)，SIREN (Sitzmann et al., 2020)，多解像度ハッシュ格子 (Müller et al., 2022) などの入力の符号化や，点の適応的な選び方といった技術が両者の間を双方向に流れた．

3.5 帰納バイアスの実現 1: 出力の形で満たす

帰納バイアスの多くは，「学習器 f_θ の後ろに，学習しない固定の写像 Φ を置く」という形で書ける．

$$u_\theta = \Phi(f_\theta).$$

Φ の形を工夫して，f_θ が何であっても (θ をどう選んでも) 出力 u_θ が条件を満たすようにする．このとき仮説空間 $\mathcal{F} = \{\Phi(f_\theta) : \theta \in \Theta\}$ は条件を満たす関数の集合の内側にあり (2.2 節の図右)，学習はその中で「どれがよいか」だけを決める．Φ が微分可能なら，損失の勾配は Φ を通って θ まで伝わるので，学習の手順はふつうの NN と変わらない．

最も簡単な例が Lagaris et al. (1998) の試行解

$$u_\theta(x) = g(x) + \phi(x)\, f_\theta(x)$$

である．g は境界条件を満たす関数，ϕ は境界で 0 になる関数とすれば，f_θ が何を出力しても境界条件は厳密に満たされる．たとえば始点の条件 $u(0) = u_0$ なら $u_\theta(t) = u_0 + t\, f_\theta(t)$，両端の条件 $u(0) = a$, $u(1) = b$ なら $u_\theta(t) = a(1-t) + bt + t(1-t)\, f_\theta(t)$ とすればよい (図)．形の複雑な領域では，境界で 0 になり内部で正になる距離関数 (の近似) を ϕ に使えば同じ形にできる (Sukumar & Srivastava, 2022)．ノイマン条件のように導関数についての条件も，g, ϕ の選び方を工夫すれば同様に課せる．

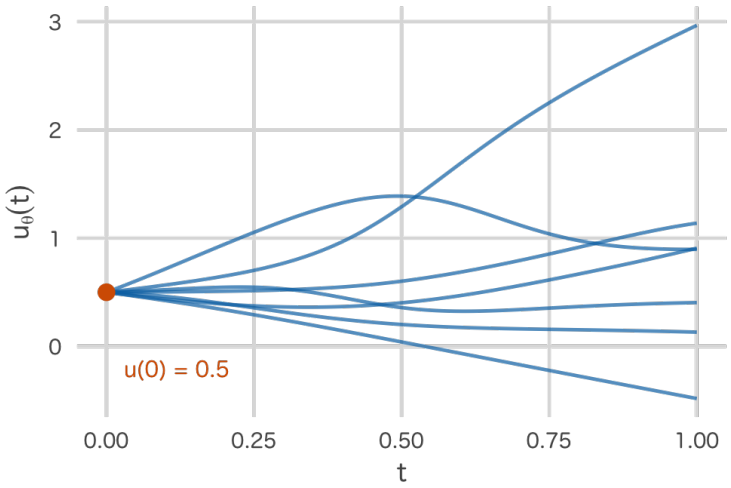


図 2: 試行解 $u_\theta(t) = 0.5 + t\, f_\theta(t)$．$f_\theta$ をいろいろ変えても始点 $u(0) = 0.5$ は動かない

4.2〜4.4 節の問題 1〜3 では，$(x, y, \varphi)(t) = t\, f_\theta(t)$ の形で始点の条件 $(x, y, \varphi)(0) = (0, 0, 0)$ を構造的に満たしている．問題 1 では始点を罰則で課した場合と比べ，この問題では差がほとんどないことを確かめる．試行解の注意点は，B の形が u_θ の導関数や勾配の大きさに影響することで，境界の近くで B が急に変わると学習が難しくなりうる．

3.6 帰納バイアスの実現 2: 積分層と射影層

試行解は境界条件を構造で満たしたが，方程式そのものはまだ罰則で課していた．方程式も構造で満たすには，「方程式を逆に解いて」条件を満たす関数を作ればよい．常微分方程式なら，学習器にはいちばん上の導関数だけを出力させ，それより下の量は積分で作る．4.5 節以降の事例で使う積分層はこの考え方による．

積分層: 曲線の例では，学習器は曲率の変化率 $j = d\kappa/dt$ だけを出力する ($f_\theta : t \mapsto j$)．格子 $t_n = nh$ ($n = 0, \dots, N-1$，$h = 1/(N-1)$) の上で，累積台形公式

$$(Tv)_0 = 0, \qquad (Tv)_n = \frac{h}{2} \sum_{m=1}^{n} (v_{m-1} + v_m) \quad (n \geq 1)$$

を表す行列を $T \in \mathbb{R}^{N \times N}$ とする．T は下三角で，第 n 行の重みは $h\,(1/2, 1, \dots, 1, 1/2, 0, \dots, 0)$ である．これを使って

$$\kappa = \kappa_0 \mathbf{1} + Tj, \qquad \varphi = \varphi_0 \mathbf{1} + LT\kappa,$$
$$x = x_0 \mathbf{1} + LT\cos\varphi, \quad y = y_0 \mathbf{1} + LT\sin\varphi$$

と順に積分する ($\cos\varphi$ などは成分ごと)．始点の条件は積分の初期値 $(x_0, y_0, \varphi_0, \kappa_0)$ で厳密に満たされ，運動学の方程式 $\varphi' = L\kappa,\ x' = L\cos\varphi,\ y' = L\sin\varphi$ は，θ が何であっても台形則の離散化の意味で厳密に成り立つ (連続の方程式に対する誤差は $O(h^2)$)．これが「積分層」，すなわち誤差逆伝播が可能な「微分可能な積分層 (累積台形公式による数値積分)」である．

T は重みが格子で決まり学習しない線形層なので，誤差逆伝播はその転置を掛けるだけで通る．損失 $\mathcal{L}$ の κ についての勾配を $g = \partial\mathcal{L}/\partial\kappa$ とすると $\partial\mathcal{L}/\partial j = T^\top g$ で，$T^\top$ は逆向きの累積和である．計算量は $O(N)$ で，積分した結果に対する損失も重み θ まで勾配が伝わる．「積分しても微分できる」ことが要点である．学習バイアスが「微分して残差を見る」のに対し，積分層は「積分して状態を作る」．入力についての自動微分は不要になり，方程式の項 $\mathcal{L}_r$ は損失から消える．微分方程式のソルバーを層として学習に組み込む Neural ODE (R. T. Q. Chen et al., 2018) や，数値計算全体を微分可能に書く微分可能プログラミング (differentiable programming) と同じ考え方である．

射影層: 始点の条件は積分の初期値で満たされるが，終点の条件 (位置・方位・曲率の 4 つ) は積分しただけでは満たされない．そこで，学習器の出力 j に低次の補正を足す．

$$\tilde{j}(t) = j(t) + \sum_{m=0}^{3} c_m\, P_m(t).$$

P_m は区間 $[0,1]$ に移したルジャンドル多項式[1]である．補正後の $\tilde{j}$ を積分層に通したときの終点の値と目標値との差を (スケールを揃えて) $F(c;j) \in \mathbb{R}^4$ とし，$F(c;j) = 0$ となる係数 $c \in \mathbb{R}^4$ を Levenberg–Marquardt 法で求める．

$$c \;\leftarrow\; c - \left(J^\top J + \mu I\right)^{-1} J^\top F(c;j), \qquad J = \frac{\partial F}{\partial c}.$$

終点の曲率 $\kappa(1)$ と方位 $\varphi(1)$ は c について線形，位置 $(x,y)(1)$ は非線形である．補正を座標の側ではなく積分の前 (曲率変化率の側) に入れるのが要点で，終点の位置を座標に直接足し込むアフィン補正では途中で $x' = L\cos\varphi$ が崩れる．本稿の実装では $\mu = 10^{-6}$ とし，学習中は 1 ステップごとに反復 6 回 (前のステップの係数から始めるウォームスタート) で解の枝が飛ばないようにし，学習後の最終の射影だけ反復 30 回とした．

射影層の勾配の通し方には 2 通りある．本稿の実装は，Levenberg–Marquardt 法の反復をすべて計算グラフに残し，自動微分でそのまま逆伝播する (反復の展開)．もう一つは，収束した解 $c^*(j)$ が $F(c^*(j);j) = 0$ を満たすことから，陰関数定理により

$$\frac{\partial c^*}{\partial j} = -\left(\frac{\partial F}{\partial c}\right)^{-1} \frac{\partial F}{\partial j}$$

で勾配を計算する方法で，反復の途中を保存しなくてよい．微分可能な射影や求解で制約を厳密に満たしながら学習する方法は Négiar et al. (2023) が扱っている．射影を学習中にも使うか推論時だけに置くかは，学習の安定性との兼ね合いで決める (4.6 節の問題 5 では推論時だけに置いた)．

[1] ルジャンドル多項式は区間 $[-1, 1]$ で直交する多項式の族，Levenberg–Marquardt 法は非線形最小二乗の反復解法 (Gauss–Newton 法と勾配法の中間) である．

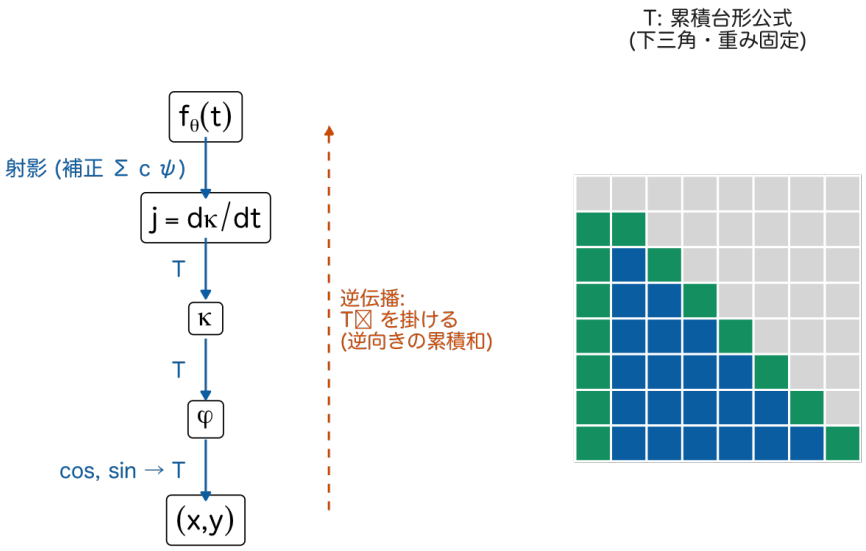


図 3: 積分層と射影層．学習器 f_θ は曲率の変化率 j だけを出力し，重み固定の下三角行列 T (累積台形公式) で積分して $\kappa, \varphi, (x, y)$ を作る．逆伝播は $T^\top$ を掛ける

積分層と射影層は，どちらも 3.5 節の $u_\theta = \Phi(f_\theta)$ の形をしている．Φ は「補正の係数を求めて足し，累積和で積分する」という固定の (非線形の) 写像で，学習するのは f_θ だけである．4.5 節の問題 4 では，罰則だけの方法は終点が 4.1 cm ずれ曲率の誤差が 79% だったのに対し，積分層と射影層を使うと終点誤差は 10^{-9} m 未満，曲率誤差は 3.4% になった．損失は滑らかさと最小半径の 2 項だけになり，終点と運動学の重みの調整が要らなくなる．

3.7 帰納バイアスの実現 3: ハミルトン力学

力学系では，保存則を構造で守る帰納バイアスがある．HNN (Hamiltonian neural network) (Greydanus et al., 2019) では，学習器 f_θ は位置 q と運動量 p を受け取ってスカラーのエネルギー (ハミルトニアン) を表し ($H_\theta(q,p) = f_\theta(q,p)$)，運動はその勾配からハミルトンの方程式

$$\dot{q} = \frac{\partial H_\theta}{\partial p}, \qquad \dot{p} = -\frac{\partial H_\theta}{\partial q}$$

で作る．勾配は自動微分で計算し，時間積分して軌道を得る．学習では，観測した $(\dot{q}, \dot{p})$ と右辺の差を損失にする．

この形の流れは H_θ の等高線に沿って進む．実際，

$$\frac{dH_\theta}{dt} = \frac{\partial H_\theta}{\partial q}\dot{q} + \frac{\partial H_\theta}{\partial p}\dot{p} = \frac{\partial H_\theta}{\partial q}\frac{\partial H_\theta}{\partial p} - \frac{\partial H_\theta}{\partial p}\frac{\partial H_\theta}{\partial q} = 0$$

である (図)．したがって，学習した H_θ が真のエネルギーと少し違っても，流れは H_θ を厳密に保存し，長時間の予測でもエネルギーがずれていかない．時間積分の離散化誤差は残るが，シンプレクティックな積分法を使えば長時間でもエネルギーの誤差は有界に保たれる．普通の NN で $(\dot{q}, \dot{p})$ を直接学ぶと，この保存が保証されず，軌道が渦を巻いて外れていくことがある．

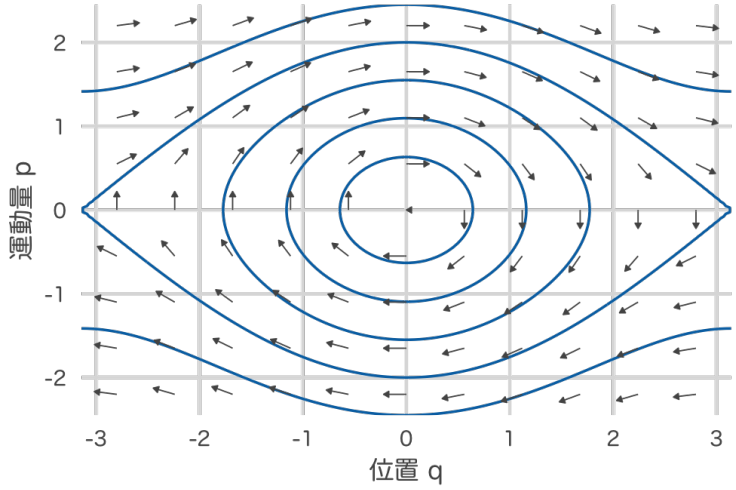


図 4: 振り子 $H = p^2/2 + 1 - \cos q$ の等高線と流れ．流れ (矢印) は H が一定の線に沿う

HNN も積分層の変種とみなせる．どちらも「学習器の出力を学習しない固定の演算に通して物理を満たす量を作る」型の帰納バイアスで，積分層は j を積分して運動学を満たす曲線を作り，HNN は H_θ をシンプレクティックな勾配 $(\partial_p H_\theta, -\partial_q H_\theta)$ に通し時間積分して保存則を満たす運動を作る．同じ型の手法には次のものがある．LNN (Cranmer et al., 2020) はラグランジアン $L_\theta(q, \dot{q})$ を学び，オイラー・ラグランジュ方程式 $\frac{d}{dt}\nabla_{\dot{q}} L_\theta = \nabla_q L_\theta$ を加速度について解いて

$$\ddot{q} = \left(\nabla_{\dot{q}} \nabla_{\dot{q}}^\top L_\theta\right)^{-1} \left(\nabla_q L_\theta - \left(\nabla_{\dot{q}} \nabla_q^\top L_\theta\right) \dot{q}\right)$$

として運動を作る．正準座標 (q, p) が使えない系にも使える．同変 NN (同変 GNN など) は，粒子系などで回転や並進に対する対称性 (同変性) を構造で保つ．3.1 節のデータ拡張が同じ対称性をデータで教えるのに対し，構造で保証する．Neural ODE (R. T. Q. Chen et al., 2018) は微分方程式の右辺を NN で表し，ソルバーを層として積分する．

3.8 関数の学習と作用素の学習

ここまでの学習器は，入力 x に対して出力 u を返す関数 $x \mapsto u_\theta(x)$ を学んでいた．PINN もこの形で，方程式の係数・境界条件・目標の条件 (まとめて条件 a と書く) を 1 つに固定し，その問題の解 u を 1 つ学ぶ．条件が変われば，最初から学び直す．4.2〜4.5 節の問題 1〜4 はこの形で，1 件に 9〜70 秒かかった．

作用素は関数を受け取って関数を返す写像である (例: 方程式の係数の関数 →解の関数)．作用素の学習では，条件 a を入力にとり，解を返す写像

$$a \longmapsto u_\theta = \mathcal{G}_\theta[a]$$

そのものを学ぶ．条件から解への写像を学ぶ NN を総称して Neural Operator (NO) と呼ぶ (Kovachki et al., 2023)．条件 a は数値の組 (4.6 節の問題 5: 終点の位置・方位と長さ) でも，関数そのもの (4.7 節の問題 6: 折れ線の方位角の関数) でもよい．学習は条件の分布全体について行うので重いが，学習後は新しい条件に 1 回の順伝播で答えられる (問題 5: 1 件 0.7 ms)．入力も出力も関数になりうる点で，普通に知られている機械学習 (ベクトルからベクトルへの写像) より一段広い枠組みである．

表 5: 関数の学習と作用素の学習

項目	関数を学ぶ (PINN)	作用素を学ぶ (NO / PINO)
学ぶ写像	$x \mapsto u_\theta(x)$	$a \mapsto \mathcal{G}_\theta[a]$
条件 a	1 つに固定	入力 (数値の組または関数)
条件が変われば	学び直し	1 回の順伝播
学習の重さ	1 件ごとに最適化	条件の分布全体で一度だけ (重い)
本稿の事例	問題 1〜4 (1 件 9〜70 秒)	問題 5 (終点の条件 → 曲率，0.7 ms)，問題 6 (折れ線 → 曲率)

作用素の学習は一般にはあまり知られていないが，「同じ形の問題を条件を変えて何度も解く」設計や制御では，1 件ごとの最適化を推論に置き換えられる．作用素の学習に物理の罰則 (学習バイアス) を加えたものが PINO で，3.10 節で PINN と PINO を一つの損失で書く．作用素を NN で近似できる根拠は，次節の作用素の万能近似定理である．

3.9 関数近似: 関数と作用素の万能近似

関数の万能近似: Cybenko (1989) はシグモイド関数，Hornik et al. (1989) はより一般の活性化関数について，1 隠れ層の NN

$$u(x) \approx \sum_{k=1}^{n} c_k \, \sigma(w_k \cdot x + b_k)$$

がコンパクト集合上の連続関数を一様に任意の精度で近似できることを示した．PINN が解 $u(x)$ を NN で表してよい理論的根拠である．ただしこれは存在定理であり，必要なユニット数や，勾配法による学習でその近似に到達できるかについては何も言わない．

近似のレート: Barron (1993) は，フーリエ変換の 1 次モーメント $C_f = \int |\omega| \, |\hat{f}(\omega)| \, d\omega$ が有限な関数について，n ユニットのシグモイド NN で半径 r の球上の 2 乗誤差を

$$\int_{B_r} \left(f(x) - f_n(x)\right)^2 \mu(dx) \le \frac{(2rC_f)^2}{n}$$

以下にできることを示した．L^2 誤差は $O(1/\sqrt{n})$ で，レートが入力次元 d によらない．格子点や多項式による近似で必要なパラメータ数が次元とともに指数的に増える次元の呪いを，このクラスの関数については避けられることが，NN が高次元の関数近似に向く理由の一つとされる．ただし評価は確率測度 μ についての平均二乗であって最悪時 (一様ノルム) の誤差界ではなく，定数 C_f は関数と次元に依存して大きくなりうる．

作用素の万能近似: 作用素 $\mathcal{G}$ は関数 a (係数，境界条件，入力の曲線など) を受け取って関数 $\mathcal{G}[a]$ (解) を返す写像である．T. Chen & Chen (1995) は，コンパクト集合上の非線形連続作用素が

$$\mathcal{G}[a](y) \approx \sum_{k=1}^{p} b_k\big(a(x_1), \dots, a(x_m)\big) \, t_k(y)$$

の形で任意の精度で近似できることを示した．出力関数を場所 y の関数 $t_k(y)$ (基底) の重ね合わせで表し，その係数 b_k を有限個のセンサー点での入力関数の値から計算する．フーリエ級数の係数を入力に応じて変えるイメージである．DeepONet (Lu, Jin, et al., 2021) はこの 2 つの部分を深層 NN にしたもので，係数を作る部分を branch net，場所の関数を作る部分を trunk net と呼ぶ．この名称は DeepONet とその派生では定着しているが，機械学習一般の用語ではない．

FNO (Li et al., 2021) は別の構造，すなわちフーリエ空間での積分作用素を層とする構造をとり，その万能近似性と誤差評価は Kovachki et al. (2021) が示した．作用素学習の一般的な枠組み (積分作用素の核を学ぶ層の積み重ね) と理論は Kovachki et al. (2023) に体系的にまとめられている．重要なのは，NO が入力関数の離散化 (格子点の数) によらない写像を表せること，すなわち粗い格子で学習して細かい格子で推論できること (解像度不変性) である．

これらの定理はいずれも「表せること」の保証であり，必要な NN の大きさや，学習で到達できるかは別の問題である．PIML の実践上の困難の多くは後者にある (6 節)．なお，ノードではなく辺に学習可能な 1 変数関数 (B-スプライン) を置く KAN (Z. Liu et al., 2025) のように，表現定理に着想を得た新しい構造も提案されているが，物理への応用で MLP に対する優位が確立したとは言えない．

3.10 PINN と PINO の統一的な定式化

PINN (physics-informed neural network) (Raissi et al., 2019) は解 $u(x)$ を NN $u_\theta(x)$ で表し，方程式・境界条件・観測のずれの重み付き和を最小化する．PINO (physics-informed neural operator) (Li et al., 2024) は，方程式の係数や境界条件・入力関数などの条件 a を NN の入力にも加え，条件から解への写像 (作用素) $\mathcal{G}_\theta$ を学

ぶ．両者は解の表し方だけが異なり，次の一つの損失で書ける．

$$\begin{aligned}\mathcal{L}(\theta,\lambda) = \mathbb{E}_a\Big[\; & w_r \frac{1}{n_r}\sum_{i=1}^{n_r}\big|\mathcal{N}[u_\theta;\lambda,a](x_i)\big|^2 && \text{方程式 } \mathcal{L}_r \\ & + w_b \frac{1}{n_b}\sum_{j=1}^{n_b}\big|\mathcal{B}[u_\theta](x_j) - g_j(a)\big|^2 && \text{境界・初期条件 } \mathcal{L}_b \\ & + w_d \frac{1}{n_d}\sum_{k=1}^{n_d}\big|u_\theta(x_k) - y_k\big|^2 \Big] && \text{観測 } \mathcal{L}_d\end{aligned}$$

ここで u_θ は，学習バイアスだけを使う場合は学習器 f_θ そのもの，帰納バイアスと組み合わせる場合は f_θ に固定の写像を施したもの $\Phi(f_\theta)$ である (3.5〜3.7 節)．帰納バイアスで満たした方程式や条件の項は損失から除く．PINN では条件 a は 1 つに固定される ($\mathbb{E}_a$ がない)．PINO では $u_\theta(x) = \mathcal{G}_\theta[a](x)$ とし，条件 a を分布から取って損失を平均する．x_i は領域内のコロケーション点，x_j は境界の点，x_k は観測点 (観測値 y_k)，w_r, w_b, w_d は各項の重みである．以下ではこの損失を $\mathcal{L} = \mathbb{E}_a[w_r\mathcal{L}_r + w_b\mathcal{L}_b + w_d\mathcal{L}_d]$ と書き，必要に応じて滑らかさなどの正則化 $\mathcal{L}_s$ や不等式制約の罰則 $\mathcal{L}_c$ を加える (4.1 節)．2.2 節の枠組みで言えば，$\mathcal{L}_d$ がデータ項，$\mathcal{L}_r$ と $\mathcal{L}_s$ が正則化項 $\mathcal{R}$ で，$\mathcal{L}_b$ は境界条件を罰則で課す場合の正則化項である．PINO の原論文 (Li et al., 2024) では，解のデータを使う教師あり項を第 3 項として加え，さらに個々の条件に対して物理の損失だけで微調整する段階を設けている．物理の損失で DeepONet を学習する physics-informed DeepONet (Wang, Wang, et al., 2021a) も同じ枠組みに入る．

3.2〜3.8 節の方法は，学習器 f_θ が系のどの関数を表し，求めたい関数 u_θ をそこからどう作るかで整理できる．どの方法でも学習するのはパラメトリックな関数 f_θ で，$u_\theta = \Phi(f_\theta)$ の Φ を選ぶのが帰納バイアス，損失に何を入れるかを選ぶのが学習バイアスである．

表 6: 学習器 f_θ が表すものと，求めたい関数 u_θ の作り方

方法	学習器 f_θ が表すもの	u_θ の作り方	本稿の事例
PINN (3.2 節)	解そのもの $x \mapsto u$	$u_\theta = f_\theta$．方程式は罰則で	—
試行解 (3.5 節)	境界からのずれ	$u_\theta = g + \phi\, f_\theta$．境界条件を形で	問題 1〜3
積分層＋射影層 (3.6 節)	導関数 u' (高階なら最高階の導関数)	f_θ を射影して積分 $\to u$	問題 4・7b
条件付き PINN (3.8 節)	条件ごとの解: $(x,a) \mapsto u$	$u_\theta = f_\theta(\cdot, a)$．問題 5 は積分層と組み合わせて u' を学ぶ	問題 5
FNO (3.8 節)	関数 $\to$ 関数: $a \mapsto u$	1 回の順伝播で解．問題 6 は u' を出して積分 (射影は推論時だけ)	問題 6
HNN (3.7 節)	エネルギー $H(q,p)$	勾配 $(\partial_p H, -\partial_q H)$ を時間積分 $\to$ 運動 $(q,p)(t)$	—

本稿の例題では，積分層の f_θ は曲率の変化率 $j = d\kappa/dt$ を表し，積分で $\kappa, \varphi, (x,y)$ を作る．問題 5 の条件 a は終点の条件，問題 6 の FNO の入力 a は折れ線の方位角で，出力は曲率 κ である．条件を入力にとるかどうか (条件付き PINN・FNO) と，学習器が何を表すか (解・ずれ・導関数・エネルギー) は別の軸である．

この定式化の利点は，メッシュを必要とせず，観測データと方程式を同じ損失に入れられることにある．逆問題やデータ同化との相性がよく，未知の物理パラメータ λ も NN のパラメータ θ と一緒に勾配法で更新すればよい．一方で，PINN は問題ごとに最適化 (学習) が必要で，同じ問題を繰り返し解く用途では古典的なソルバーより遅いのが普通である．PINO は学習が重いが推論は 1 回の順伝播で済む．どちらも重みの選択に結果が左右される．

3.11 何を求めるか: 順問題・設計問題・逆問題

同じ損失の形のまま，何が既知で何を求めるかによって 3 種類の問題になる．

表 7: 既知と未知の組合せによる問題の分類

問題	分かっているもの	求めるもの (PINN)	損失に入れる項	PINO の条件 a
順問題	方程式・物理パラメータ λ・境界条件	解 u	方程式＋境界	物理パラメータ・境界条件・入力の関数
設計問題	方程式の形・目標とする条件	解 u と設計変数 λ	方程式＋目標の条件	目標の条件
逆問題 (同定)	方程式の形・観測データ	解 u と未知の物理パラメータ λ	方程式＋観測	観測データ

PINN では，未知の λ を NN のパラメータ θ と一緒に勾配法で更新するだけで，どの問題も同じ実装で扱える．これが PINN の大きな魅力である．同じ問題を条件を変えながら何度も解きたいときは，表の「分かっているもの」を条件 a として作用素の入力にとり，$a \mapsto (u, \lambda)$ をまとめて学ぶ．4 節の問題 5 (終点の条件 → 曲率) と問題 6 (折れ線 → 曲率) は，設計問題をこの PINO 型で解いたものである．

4 事例研究: 制約付き曲線の設計

本節では，道路や車両の経路の設計に現れる曲線を題材に，問題を少しずつ難しくしながら PIML の技術がどこで必要になるかを確かめる．すべての問題を同じ表 (学習器，学習する量，方程式・境界・データの損失) で定式化し，数値実験の結果とその意味を示す．数値はすべて CPU (float64) での実測である．

4.1 題材と共通の記法

曲線の形を考えるときは，時刻ではなく，基準点から曲線に沿って測った距離 (弧長) s を変数にとる．弧長 s の点の位置を $(x(s), y(s))$，進行方向が x 軸となす角 (方位角) を $\varphi(s)$，曲がり具合 (曲率) を $\kappa(s) = 1/R$ (R は曲率円の半径，左曲がりを正) とすると，曲線は次の連立常微分方程式で決まる．

$$\frac{dx}{ds} = \cos\varphi, \qquad \frac{dy}{ds} = \sin\varphi, \qquad \frac{d\varphi}{ds} = \kappa(s).$$

曲率 $\kappa(s)$ が入力 (運転ではハンドル操作) で，位置と方位 (x, y, φ) が出力 (軌跡) にあたる．二輪モデルでは舵角 δ とホイールベース L_w を用いて $\kappa = \tan\delta / L_w \approx \delta / L_w$ で，曲率は舵角にほぼ比例する．本節の「物理」は，横滑りを無視したこの運動学の方程式を指す．

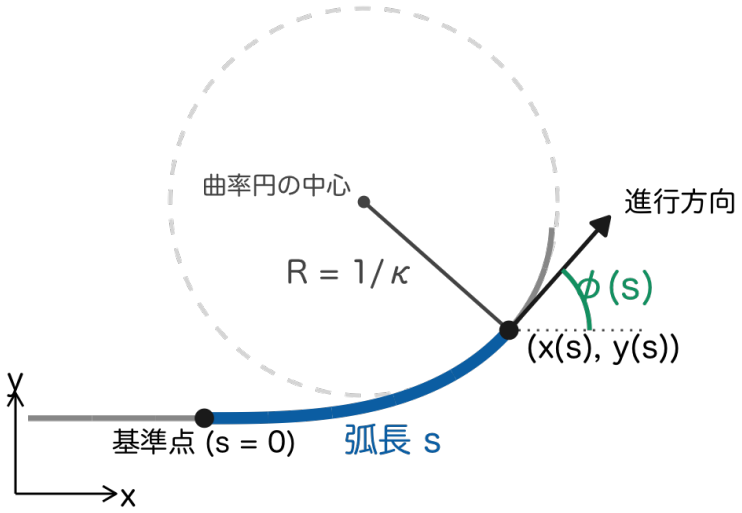


図 5: 弧長 s，位置 $(x(s), y(s))$，方位角 $\varphi(s)$，曲率 $\kappa = 1/R$ の定義

曲率が 0 なら直線，一定なら円弧，弧長に比例する $(\kappa = s/A^2)$ 曲線がクロソイドである．クロソイドは道路で直線と円弧をつなぐ緩和曲線として使われ，半径 R の円弧に長さ L で接続するには $A^2 = RL$ とする．等速 v で走ると横加速度は $a_y = v^2\kappa$，横躍度 (横ジャーク．横加速度の変化率) は $j_y = v^3\, d\kappa/ds$ で，クロソイド区間では横躍度が一定になり乗員が感じる横向きの力が一定の割合で増える．これが緩和曲線を入れる力学的な理由である．クロソイドの座標 $x(s) = \int_0^s \cos(u^2/2A^2)\, du$ はフレネル積分で，初等関数では表せない．最も単純な場合でも，経路を求めるには数値的に積分する必要がある．

以下の定式化では次の記法を共通に用いる．弧長を区間長 L で無次元化して $t = s/L \in [0,1]$，$' = d/dt$ とする．NN の入力の範囲を揃えると学習が安定するためで，$ds = L\, dt$ から方程式の各成分に L が掛かる．損失は次の 5 つの項の重み付き和

$$\mathcal{L} = \mathcal{L}_r + w_b\mathcal{L}_b + w_d\mathcal{L}_d + w_s\mathcal{L}_s + w_c\mathcal{L}_c$$

の一部として書き，問題ごとにどの項を使い，重み w をいくつにしたかを定式化の表で示す．各項は次のとおりである．

$$r(t) = (x' - L\cos\varphi,\ y' - L\sin\varphi,\ \varphi' - L\kappa) \qquad \text{方程式の残差}$$

$$\mathcal{L}_r = \frac{1}{n_r}\sum_{i=1}^{n_r} \big|r(t_i)\big|^2 \qquad \text{方程式}$$

$$\mathcal{L}_b = \big|(x,y,\varphi)(1) - (x_1,y_1,\varphi_1)\big|^2 \qquad \text{境界 (終点)}$$

$$\mathcal{L}_d = \frac{1}{n_d}\sum_{k=1}^{n_d} \big|(x,y)(t_k) - (x_k,y_k)\big|^2 \qquad \text{データ}$$

$$\mathcal{L}_s = \frac{1}{n_r}\sum_{i=1}^{n_r} \big|\kappa''(t_i)\big| \approx \int_0^1 |\kappa''|\, dt \qquad \text{滑らかさ (L1)}$$

$$\mathcal{L}_c = \frac{1}{n_r}\sum_{i=1}^{n_r} \mathrm{ReLU}(g(t_i))^2 \qquad \text{制約 } (g \le 0 \text{ の罰則})$$

t_i は区間内のコロケーション点 $(n_r = 128)$，$\mathrm{ReLU}(z) = \max(0,z)$ である．$\mathcal{L}_r, \mathcal{L}_b, \mathcal{L}_d$ は 3.10 節の一般形の 3 項にあたり，始点は解を $t\, f_\theta(t)$ の形にして構造で満たす (3.5 節の試行解) ので，$\mathcal{L}_b$ に入れるのは終点だけである．$\mathcal{L}_s$ は 2.2 節の正則化項 $\mathcal{R}$ にあたり (2.3 節の ℓ_1 トレンドフィルタと同じ形)，絶対値の和 (L1) は κ''

が所々にだけ現れる解，すなわち区分線形の曲率 (クロソイドと円弧の組合せ) を好む．$\mathcal{L}_c$ は不等式 $g \le 0$ を超えた分だけを罰する項で，問題 4 の最小半径と問題 7b の横加速度・横躍度の上限に使う．

表 8: 8 つの問題の構成 (「難しさ」は一つ前までの方法で困ること)

問題	種類	与えるもの → 求めるもの	難しさ	新しい技術
1	順問題	曲率と始点 → 経路	—	PINN の基本形
2	設計	終点 → 曲率の係数と長さ	未知の係数がある	係数も NN と同時に学習
3	同定	前半 5 点の測量 → 係数と全体の経路	観測が一部の区間にしかない	データの項と，その重みの調整
4	曲率分布の設計	両端の条件 → 曲率の関数	未知が関数．罰則では端と方程式がずれる	積分層と射影 (構造で満たす)，L1
5	条件付き	終点の条件 → 曲率 (何度も)	1 件ごとに数十秒かかる	条件付き PINN
6	作用素学習	折れ線 (関数) → 滑らかな経路	入力が関数で，点数もまちまち	FNO / PINO
7a	速度	曲線 → 速度	曲線だけでは横加速度などが上限超え	凸最適化 (SOCP，逐次凸化)
7b	速度と幾何	一定の速度 → 走れる曲線	7a では減速が要る	動特性の罰則

4.2 問題 1: 順問題

問題: 曲率 $\kappa(s) = s/(RL)$ と始点 $(x, y, \varphi)(0) = (0, 0, 0)$ が与えられたとき，経路に沿った弧長 s の関数として経路 $(x(s), y(s), \varphi(s))$ を求める．ここでは $R = 150$ m，$L = 80$ m とする ($A = 109.5$ m のクロソイド)．答えは数値積分で一つに定まるので，PINN が方程式を解けることの確認にあたる．

表 9: 問題 1 の定式化

項目	内容
学習器	MLP f_θ: $t \mapsto (x, y, \varphi)$．解は $(x, y, \varphi)(t) = t\, f_\theta(t)$
学習する量	NN のパラメータ θ
方程式 $\mathcal{L}_r$	曲率 $\kappa = t/R$
境界 $\mathcal{L}_b$	始点は $\times t$ で構造的に満たす (ハード)．比較: 始点のずれ $\lvert(x, y, \varphi)(0)\rvert^2$ を罰則に (ソフト)
データ $\mathcal{L}_d$	なし
損失	$\mathcal{L} = \mathcal{L}_r$ (ソフトは $\mathcal{L}_r + w_b\mathcal{L}_b$)，$w_b = 10$

NN は中間層 3 層・各 32 ユニット (活性化関数 tanh) の MLP で，解を $(x, y, \varphi)(t) = t\, f_\theta(t)$ の形にすると，NN が何を出力しても始点条件が満たされる (ハード制約)．比較として，出力をそのまま解とし始点条件を罰則で加える版 (ソフト制約) も試した．最適化は Adam 3,000 回 (学習率 3×10^{-3}) の後に L-BFGS 300 回である．

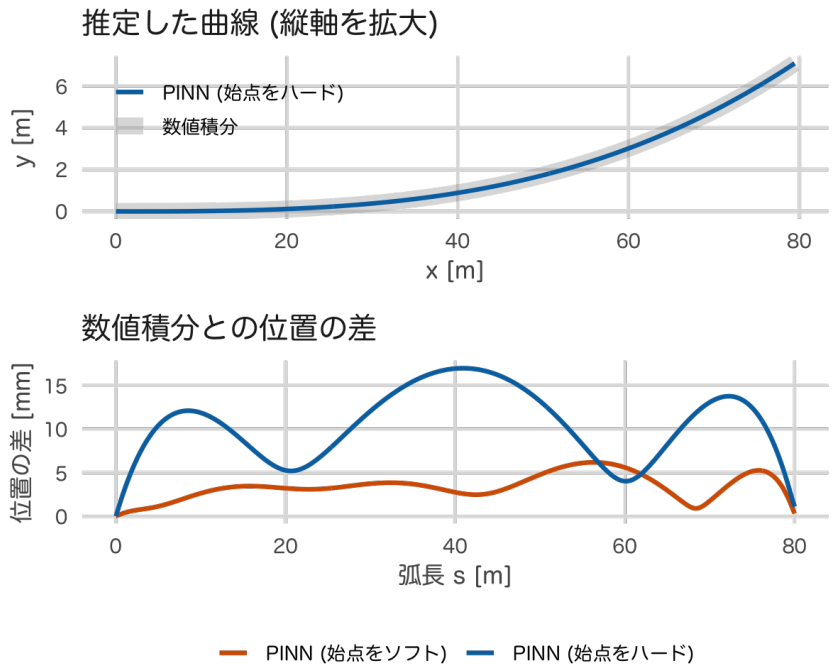


図 6: 問題 1: PINN の曲線と数値積分の曲線 (上)，位置の差 (下)

結果: 乱数シードを 5 通り変えて学習した．数値積分 (台形則，0.2 m 刻み) の解との位置の差の最大は 6.4〜17.0 mm (ハード)，3.3〜9.1 mm (ソフト) で，80 m の区間に対して相対誤差 10^{-4} 程度である．差は区間の中で波打っており，NN が方程式を満たしきれない分 (近似と最適化の誤差) がそのまま位置に出ている．その大きさは最適化がどこまで詰めるかで決まるので，初期値によって 2〜3 倍変わり，計算機やライブラリの版によっても変わる．始点の誤差はハードで 0，ソフトで 0.05 mm 以下で，この問題では課し方による差は小さい．学習に 1 回約 4 秒かかるのに対し，台形則は同じ答えを 0.3 ms，誤差 0.02 mm で出す．

意味: 方程式も入力もすべて分かっている順問題だけなら，PINN を使う理由はない．数値積分は始点から刻みごとに順に進むが，PINN は区間全体の関数を一度に表し，すべての点で方程式を満たすように全体を同時に調整する．この枠組みの利点が出るのは，未知のものが増えたときである．

4.3 問題 2: 設計

問題: 曲率が弧長に比例して変わる曲線 $\kappa = \kappa_0 + (\kappa_1 - \kappa_0)s/L$ について，始点 $(0, 0, 0)$ と終点の位置・方位 (x_1, y_1, φ_1) が与えられたとき，両者を結ぶ曲線のパラメータ (κ_0, κ_1, L) を求める．端点の位置と向き (接線) を指定して曲線で結ぶので G1 エルミート補間と呼ぶ．G1 は幾何学的連続の 1 次で，曲線どうしが位置と接線の向きをそろえてつながることを指す (G0 は位置だけ，G2 は曲率までそろう)．以下，問題 1 の曲線の終点を与えた場合を「緩和曲線」，任意の終点を与えた場合を「一般の G1」と呼ぶ．条件は 3 つ，未知数も 3 つである．古典的にはフレネル積分で終点を表し Newton 法で解く (Bertolazzi & Frego, 2015)．本稿では ODE の数値積分と Newton 法によるシューティングで参照解を作った．

表 10: 問題 2 の定式化

項目	内容
学習器	MLP f_θ: $t \mapsto (x, y, \varphi)$ (問題 1 と同じ)
学習する量	NN のパラメータ θ と物理パラメータ $\lambda = (\kappa_0, \kappa_1, L)$ (設計変数)
方程式 $\mathcal{L}_r$	曲率 $\kappa = \kappa_0 + (\kappa_1 - \kappa_0)t$
境界 $\mathcal{L}_b$	始点: 構造．終点: 目標 (x_1, y_1, φ_1) とのずれ
データ $\mathcal{L}_d$	なし
損失	$\mathcal{L} = \mathcal{L}_r + w_b\mathcal{L}_b$，$w_b = 10$

長さは終点までの直線距離で無次元化し，L は正値を保つため $\log L$ を学習する．終点条件の重み $w_b = 10$ は試行で決めた．古典的なシューティング法は「ODE を積分して終点のずれを見る」内側と「Newton 法でパラメータを直す」外側の 2 重ループになるが，PINN では両方を 1 つの勾配法で同時に解く．

表 11: 問題 2: 終端条件だけから求めたパラメータ

ケース	手法	κ_0 [1/m]	κ_1 [1/m]	L [m]	最大乖離 [m]
緩和曲線	古典解	0	0.0066667	80.000	—
緩和曲線	PINN	2.7e-07	0.0066662	80.000	0.012
一般の G1	古典解	0.0018108	0.0167232	113.003	—
一般の G1	PINN	0.0018113	0.0167226	113.003	0.008

結果: 緩和曲線 (終端条件 $(79.4, 7.1)$ m，15.3°，すなわち問題 1 の曲線の終点) では，PINN は $\kappa_0 \approx 2 \times 10^{-7}$，$\kappa_1 = 0.0066663$ (真値 $1/150 = 0.0066667$)，$L = 80.000$ を得た (乱数シード 0．表)．終端条件だけから「始点で曲率 0 の緩和曲線であること」と R, L を復元したことになる．一般の G1 ($(100, 40)$ m，60°) でも κ_1 は古典解と 4 桁，L は 5 桁一致した ($L = 113.001$〜113.003 m)．乱数シードを 5 通り変えると，古典解の曲線との最大乖離は緩和曲線で 7〜28 mm，一般の G1 で 8〜16 mm で，緩和曲線の 1 試行だけ κ_1 が 0.4% ずれた．計算時間は 1 ケース約 5 秒，シューティング法は 0.1 秒未満である．

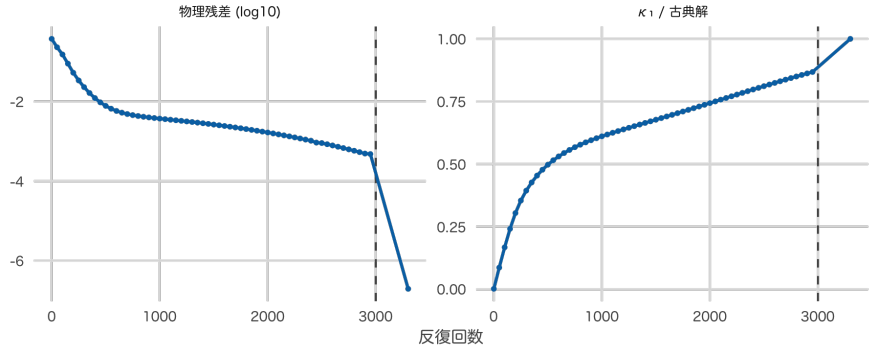


図 7: 問題 2 (一般の G1) の学習の経過．破線で Adam から L-BFGS に切り替えた

学習の経過を見ると，Adam の 3,000 回で κ_1 は古典解の 0.87 倍程度までゆっくり増え (2,700 回の時点で 84%)，物理残差は 10^{-3} 程度で停滞する．L-BFGS に切り替えた直後に κ_1 は古典解に収束し，残差は 10^{-7} 程度まで下がった．6 節で述べる最適化の悪条件性の小さな実例である．

意味: 物理パラメータ (設計変数) を NN のパラメータ θ と一緒に勾配法で更新するだけで設計問題が解ける．ただし未知数が 3 つ程度なら古典解法の方が圧倒的に速い．

4.4 問題 3: 同定

問題: 曲率が弧長に比例する既設の緩和曲線 $\kappa(s) = s/A^2$ の始点側で，弧長 s_k の n_d 点の座標を測量した (測量誤差の標準偏差 σ)．このとき，パラメータ A を推定し，未測量の区間を含む長さ L の経路全体を再現する．ここでは $s_k = 0, 10, \ldots, 40$ m ($n_d = 5$)，$\sigma = 5$ cm，$L = 80$ m とし，真値は $A = 109.54$ m である．観測のない区間をどう外挿するかが問題になる．

表 12: 問題 3 の定式化

項目	内容
学習器	MLP f_θ: $t \mapsto (x, y, \varphi)$ (問題 1 と同じ)
学習する量	NN のパラメータ θ と物理パラメータ A (未知の係数)
方程式 $\mathcal{L}_r$	曲率 $\kappa = tL/A^2$
境界 $\mathcal{L}_b$	始点: 構造 (終点の条件はない)
データ $\mathcal{L}_d$	測点 s_k (n_d 点) での位置のずれ
損失	$\mathcal{L} = \mathcal{L}_r + w_d \mathcal{L}_d$，$w_d = 10, 100, 1000$

比較として，(a) 方程式の項を外しデータだけに合わせた NN ($\mathcal{L} = \mathcal{L}_d$)，(b) A を変えながら方程式を数値積分し n_d 点との差の 2 乗和を最小にする非線形最小二乗，を用いた．重み w_d は 10，100，1000 の 3 通り，乱数シード (測量誤差と初期値) は 5 通り試した．最適化は Adam 4,000 回 → L-BFGS 300 回である．

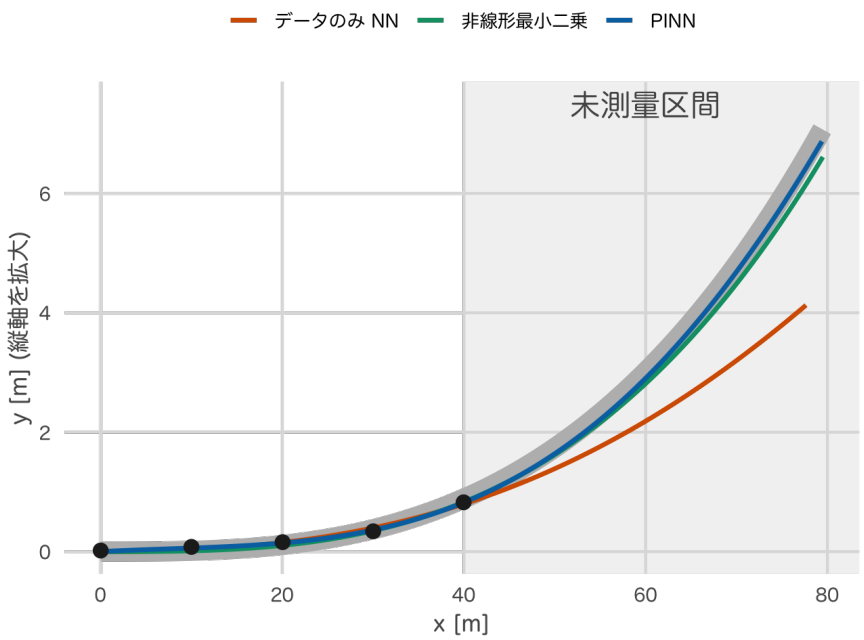


図 8: 問題 3: 測点 0〜40 m の 5 点から 80 m まで再現した曲線 (乱数シード 0，$w_d = 100$．灰色の帯は真の曲線)

結果: 全区間の位置誤差の最大値 (5 試行) は，データのみの NN が 2.9〜3.7 m で，A は推定できない．PINN ($w_d = 100$) は 0.17〜1.72 m で $A = 98.3$〜110.9 m，非線形最小二乗は 0.12〜0.98 m で $A = 102.7$〜113.4 m だった．5 試行のうち 3 試行では両者とも 0.5 m 以下で同程度だが，残る 2 試行では PINN が 0.9〜1.7 m と，非線形最小二乗より大きく外れた．5 点の測量から 40 m 先を外挿する問題はそれ自体が測量誤差に敏感で，未知数が 1 つなら古典手法の方が安定している．

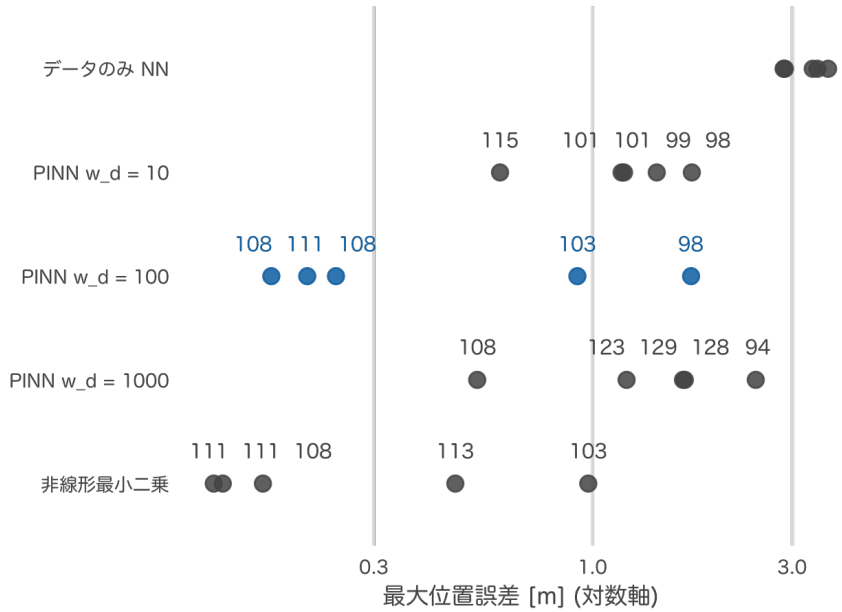


図 9: 問題 3: データ項の重みと最大位置誤差．点は乱数シード 5 通り，数字は推定した A [m]

重みを変えると結果は大きく変わる．$w_d = 10$ では物理残差の比重が大きくデータへの当てはまりが甘くなり 0.60〜1.73 m (A は 98〜115 m とばらつく)．$w_d = 1000$ では 5 試行中 3 試行が $A \approx 122$〜129 m の局所解に落ちて 1.2〜1.7 m になり，測量誤差の悪い 1 試行は $A = 94$ m で 2.5 m 外れた (残る 1 試行は 0.53 m)．データ項が強すぎると学習初期に NN がデータ点だけに合わせてしまい，物理と矛盾した状態から抜け出せなくなると考えられる．

意味: 物理残差は「観測のない区間でも方程式を満たせ」という制約で，外挿を支える．これは 2.2 節の学習の枠組みで言う学習バイアス (正則化としての物理) の具体例であり，重み w_d の選択は正則化パラメータの選択そのものである．一方，未知数が 1 つで方程式の形が既知なら，古典的な非線形最小二乗の方が速く，測量誤差の悪い試行でも外れ方が小さい (安定している)．PINN の利点が出るのは，曲率分布そのものが未知の関数である場合や，他の物理モデルと連成する場合である．

4.5 問題 4: 曲率分布の設計

問題: 長さ L の曲線について，始点と終点の位置・方位・曲率 $(x, y, \varphi, \kappa)(0) = (0, 0, 0, 0)$，$(x, y, \varphi, \kappa)(L) = (x_f, y_f, \varphi_f, 0)$ が与えられたとき，これらと最小半径 $R_{\min}$ ($|\kappa(s)| \le 1/R_{\min}$) を守り，曲率の変化が最も少ない ($\int_0^L |\kappa''(s)|\, ds$ が最小の) 曲率 $\kappa(s)$ と経路を求める．ここでは $L = 110$ m，$(x_f, y_f, \varphi_f) = (100.1\text{ m}, 36.5\text{ m}, 0.70\text{ rad})$，$R_{\min} = 100$ m とする．問題 2 では曲率の形を線形と決めていたが，ここでは曲率そのものが自由な関数になる．終点の条件は「クロソイド 40 m・円弧 30 m (半径 100 m)・クロソイド 40 m」の複合曲線から作ったので，これが答え合わせ用の真値になる．区間の構成を教えずに複合曲線を見つけられるかが問われる．

表 13: 問題 4 の定式化 (左: 問題 2 の方法をそのまま使う罰則版，右: 物理を構造に移す版)

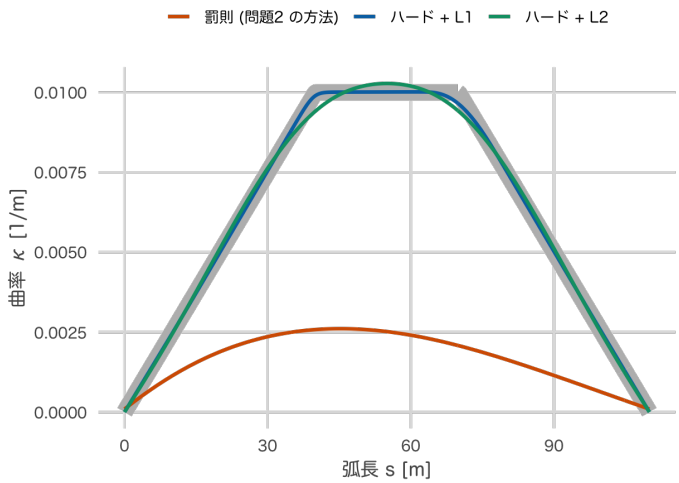
項目	罰則 (問題 2 の方法)	構造 (ハード制約)
学習器	MLP f_θ: $t \mapsto (x, y, \varphi, \kappa)$	MLP f_θ: $t \mapsto j$ ($j = d\kappa/dt$)．後ろに積分層 $j \to \kappa \to \varphi \to (x, y)$ と射影
学習する量	NN のパラメータ θ	NN のパラメータ θ
方程式 $\mathcal{L}_r$	使う	0 (積分層で構造的に満たす)
境界 $\mathcal{L}_b$	始点・終点 (各 4 条件)	射影で厳密に満たす
データ $\mathcal{L}_d$	なし	なし
滑らかさ $\mathcal{L}_s$	使う	使う
制約 $\mathcal{L}_c$	最小半径 $g = R_{\min}\lvert\kappa\rvert - 1$	同左
損失	$\mathcal{L}_r + w_b\mathcal{L}_b + w_s\mathcal{L}_s + w_c\mathcal{L}_c$	$\mathcal{L}_s + w_c\mathcal{L}_c$

構造版では，3.6 節の積分層と射影層を使う．NN (中間層 3 層・各 64 ユニット，tanh，最終層を 0 で初期化) は曲率変化率 $j = d\kappa/dt$ だけを出力し，台形則の累積和で $\kappa \to \varphi \to (x, y)$ を作る．運動学の残差は離散化誤差を除いて 0 になる．終点の 4 条件 $(x, y, \varphi, \kappa)(1)$ は，j にルジャンドル多項式 4 本の補正 $\sum_m c_m P_m(t)$ を足し，係数 c を Levenberg–Marquardt 法で決めて厳密に満たす．射影は反復を展開して微分するので，NN は射影後の曲線に対する損失で学習できる．残る損失は滑らかさ $\mathcal{L}_s$ と最小半径の罰則 $\mathcal{L}_c$ ($g = R_{\min}|\kappa| - 1$) の 2 項だけである．L1 ノルムは κ'' を疎に (ほとんど 0 で所々に集中) させるので，曲率が区分線形，すなわちクロソイドと円弧の組合せになりやすい (2.3 節)．L2 ($\int \kappa''^2 ds$) は全体に小さく広げるので角が丸くなる．罰則版の重みは $(w_b, w_s, w_c) = (10, 10^{-4}, 10^5)$ とした．

結果: 罰則版 (問題 2 の方法) は終点が 4.1 cm，方位が 0.9° ずれ，曲率は真値から最大 79% ずれた (最小半径の罰則の重みを変えても 16%)．ピークは真値の 1/4 程度しかなく，形が大きく崩れる．構造版は終点誤差が 10^{-9} m 未満 (浮動小数点の精度) で，曲率誤差は L1 で 3.4%，L2 で 5.7% だった．L1 は台形の折れ点まで再現し，L2 は角が丸い．計算時間は 1 件約 70 秒である．区間の構成を知っていれば，古典的な最小二乗 (クロソイド長と最大曲率を求める) で 0.2 秒で解ける．

意味: 未知が関数になると，4 つの罰則項の重みのつり合いで解が決まるため，「どの項をどれだけ満たすか」の取引きで形が定まらない．物理 (運動学) と条件 (終点) を構造に移すと，損失は滑らかさと最小半径の 2 項になり，終点と運動学の重みの調整なしに形が決まる．これは損失の重み付けの問題を，重み付けの工夫ではなく問題の設計で消す解決法とも言える．

曲線の図と曲率の図: 位置は曲率を 2 回積分した量なので，曲率の細かな違いは位置にはほとんど残らない．構造版では両端の位置・方位・曲率が射影で最初から固定されるため，学習器の出力が 0 の学習前の曲線でも，真の曲線との差は最大 1.5 cm 程度にとどまる (曲率の差は最大 6×10^{-4} 1/m で，真の最大曲率の 6%．デモ



(a) 求めた曲率分布 (灰色の帯は真値)

手法	終点 [m]	κ 誤差
罰則	0.041	79.2%
ハード+L1	< 1e-9	3.4%
ハード+L2	< 1e-9	5.7%

(b) 終点誤差と曲率の誤差 (曲率誤差は真値との差の最大値を最大曲率で割った値)

図 10: 問題 4 の結果

demo/clothoid-pinn.html の設定で計算)．問題 4 は通る道筋ではなく曲率の配り方 (ハンドル操作の滑らかさ) を設計する問題であり，学習の効果は平面図ではなく曲率の図に現れる．

4.6 問題 5: 条件を変えて何度も解く

問題: 実際の設計や経路計画では，入口と出口の条件を変えながら同じ問題を何度も解く．問題 4 の終点の条件 a がさまざまに与えられたとき，どの $a = (x_f, y_f, \varphi_f, L)$ に対しても問題 4 の解を短い時間 τ で返す写像 $c \mapsto \kappa(\cdot\,; a)$ を求める．ここでは $\tau \approx 1$ ms (1 件あたり) を目標とする．

表 14: 問題 5 の定式化

項目	内容
学習器	MLP f_θ: $(t, a) \mapsto j$ (問題 4 と同じ j)．後ろに積分層．射影は推論時だけ
学習する量	NN のパラメータ θ (条件 $a = (\varphi_f,\, x_f/L,\, y_f/L,\, L/100)$ は入力)
方程式 $\mathcal{L}_r$	0 (積分層で構造的に満たす)
境界 $\mathcal{L}_b$	学習中: 罰則．推論時: 射影で厳密に
データ $\mathcal{L}_d$	なし (曲率の正解は使わない)
滑らかさ $\mathcal{L}_s$	使う
損失	$\mathcal{L} = \mathbb{E}_a[\mathcal{L}_s + w_b \mathcal{L}_b]$ (条件の分布で平均)，$w_b = 1000$

学習用の条件は，3 区間の複合クロソイド (クロソイド長 25〜50 m，円弧長 10〜40 m，最小半径 80〜150 m) をランダムに作って得た．曲率の正解は教師として使わず，損失は滑らかさと終点の罰則だけである．学習中から射影を入れると，射影の反復が条件によって収束しにくく勾配が不安定になって学習が進まなかったため，学習中は終点を罰則で近似的に満たし，推論時だけ射影で厳密に一致させる分担にした．3.10 節の PINO の式で，条件 a が有限個の数 a になった場合にあたる．

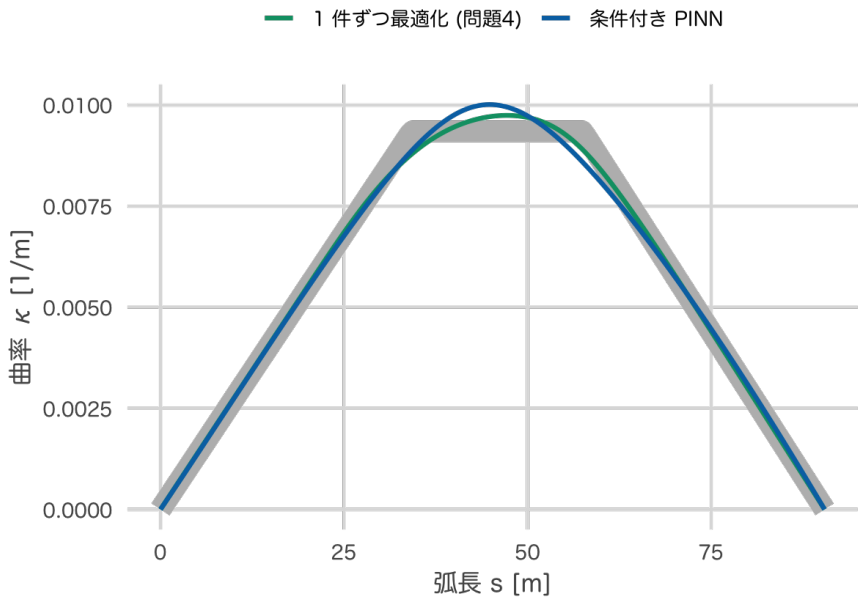


図 11: 問題 5: 未学習の条件での曲率の例 (灰色の帯は真値)．条件付き PINN は 1 回の順伝播と射影で解を返す

結果: 未学習の条件 200 件で，推論 0.27 ms と射影 0.46 ms の計 0.7 ms/件で解が得られた．射影前の終点誤差は中央値 0.79 m，90% 点 1.86 m で，射影後はすべて 10^{-13} m 以下になった．1 件ずつ最適化する問題 4 の方法は約 25 秒/件で，その解の方が滑らかさ (L1 の値) は 1.05〜1.5 倍よい．学習は 4,000 ステップ，約 230 秒で，一度だけでよい．

意味:「条件 → 解」の写像を前もって学べば，1 件ごとの最適化が不要になり，計算は 4 桁以上速くなる．代償は解の最適性のわずかな低下である．射影を推論時に置くことで，学習の安定性と条件の厳密な充足を両立できる．

4.7 問題 6: 作用素学習

問題: 手書きのスケッチや探索アルゴリズムで得たカクカクの暫定経路 (折れ線) を，車が走れる滑らかな経路に直したい．折れ線の方位角の関数 $\varphi_{\mathrm{raw}}(s)$ が与えられたとき，その近くを通り，始点と終点の位置・方位が一致する滑らかな経路の曲率を求める．ただし，形も点数も異なる折れ線に対して，一つの写像 $\varphi_{\mathrm{raw}} \mapsto \kappa$ で答える．入力は数個の数ではなく関数そのもので，解像度もまちまちである．

表 15: 問題 6 の定式化

項目	内容
学習器	FNO $\mathcal{G}_\theta$: $\varphi_{\mathrm{raw}}(\cdot) \mapsto \tilde{\kappa}(\cdot) = L\kappa(\cdot)$ (等間隔の格子 N 点) ．後ろに積分層．射影は推論時だけ
学習する量	NN のパラメータ θ (折れ線 φ_{raw} は入力の関数)
方程式 $\mathcal{L}_r$	0 (積分層で構造的に満たす)
境界 $\mathcal{L}_b$	学習中: 罰則．推論時: 射影で厳密に
データ $\mathcal{L}_d$	折れ線 $(x, y)_{\mathrm{raw}}$ からのずれ (正解の経路は使わない)
滑らかさ $\mathcal{L}_s$	使う
損失	$\mathcal{L} = \mathbb{E}_{\varphi_{\mathrm{raw}}}[\mathcal{L}_d + w_s\mathcal{L}_s + w_b\mathcal{L}_b]$，$(w_s, w_b) = (5 \times 10^{-6}, 10)$

学習用の折れ線は 2 回曲がるもの (曲がる角 20〜60°，区間長 30〜70 m) をランダムに作り，正解の滑らかな経路は与えない (教師なし)．FNO は幅 32，16 モード，4 層で，出力は無次元曲率 $\tilde{\kappa} = L\kappa$，その差分を積分層に渡す．FNO の重みは格子点ではなくフーリエモード (周波数) ごとに付くので，同じ関数を等間隔の格子の何点で読んでも低いモードの係数はほぼ同じで，同じ重みがどの点数 N にも使える．折れ線の節点 (曲がる所) の数は関係なく，方位角の関数を等間隔の N 点で読む (FFT のため等間隔が要る)．学習は 6,000 ステップ，約 170 秒である．重みは表の (w_s, w_b) で，実装では損失全体を 10^3 倍している．

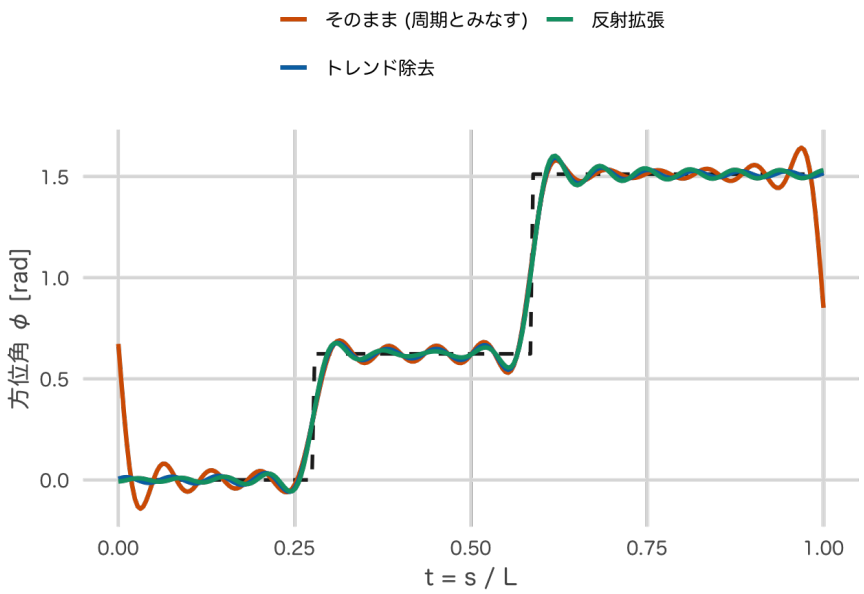
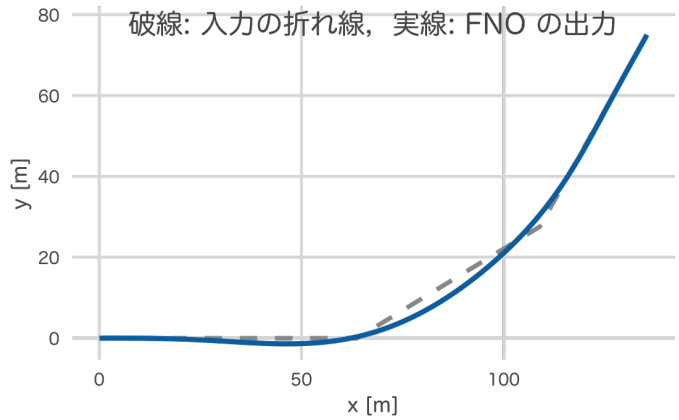


図 12: 折れ線の方位角を 16 モードで打ち切ったフーリエ級数で表した例

フーリエ層の周期性の影響をまず確かめる．2 回曲がる折れ線の方位角 (段差の合計 1.51 rad) を低次 16 モードで打ち切ったフーリエ級数で表すと，周期関数とみなした場合は端で 0.67 rad (段差の約 45%) の誤差が出た．トレンド除去では 0.015 rad，反射拡張 (同じ解像度にするため 32 モード) では 0.020 rad で，どちらも端の誤差を 1/30 以下にした．



(a) $N = 128$ で学習し $N = 256$ で推論した例 (破線: 入力の折れ線，実線: 出力)

N	ずれ [m]	端処理あり	ms/件
128	1.46	2.10	0.27
256	1.45	2.08	0.30
512	1.45	2.08	1.56

(b) 未学習の折れ線 200 本での折れ線からのずれ (中央値) と推論時間

図 13: 問題 6 の結果

結果: $N = 128$ 点だけで学習した NN で，未学習の折れ線 200 本に対する折れ線からのずれ (RMS 距離の中央値) は，$N = 128, 256, 512$ で 1.46，1.45，1.45 m とほぼ同じだった (解像度不変)．意外なことに，端の処理 (トレンド除去とゼロ・パディング 25%) を入れた版の方が 2.08〜2.10 m と悪かった．射影の失敗 (終点に届かない無理な折れ線) は端の処理なしで 3〜4%，ありで 1% 前後である．

意味: 出力を曲率にしており，道路の始点・終点は直線なので曲率は両端で 0 に近い．周期とみなしても出力側には段差がなく，この課題ではギブス現象が問題にならなかったと考えられる．入力の方位角の段差は残るが，FNO の各層にある点ごとの線形変換が周期でない成分を運ぶ．トレンド除去は入力の情報 (終点の方位) を一部消すので不利に働いた可能性がある．「端の処理は常に必要」なのではなく，「何をフーリエ層に通すか (何を出力にするか)」で必要性が変わる，というのが実践上の教訓である．

4.8 速度と幾何: 走れる軌道 (問題 7a・7b)

ここまでは幾何 (曲率分布) だけを扱った. 車両がその軌道を実際に走れるかは速度による. 横加速度は $a_y = v^2\kappa$, 横躍度は $j_y = v^3\, d\kappa/ds + 2v\, a_t\, \kappa$ ($a_t = v\, dv/ds$ は接線加速度) である. 問題 4 の幾何 (最小半径 100 m) を 20 m/s で走ると $a_y = 4.0$ m/s², $j_y = 2.0$ m/s³ となり, 上限 (横加速度 $\bar{a}_y = 3.0$ m/s², 摩擦円 $\bar{a} = 3.5$ m/s², 横躍度 $\bar{j}_y = 0.8$ m/s³) を超える. 摩擦円はタイヤが出せる加速度 (前後と横の合成) の上限, 横躍度は横加速度の変化率で乗り心地に効く. 対処は 2 通りある. 幾何を固定して速度を最適化する分離型 (7a) と, 速度を保ったまま動特性の罰則で幾何そのものを変える統合型 (7b) である.

4.8.1 問題 7a: 決まった曲線を走る速度

進入直線 (長さ L_{in})・問題 4 の曲線 (長さ L)・出口直線 (長さ L_{out}) をつないだ経路の曲率と, 進入速度 v_0, 加速度と躍度の上限が与えられたとき, これらを守り走行時間 T が最短となる速度 $v(s)$ を求める.

$$\min_{v}\ T = \int \frac{ds}{v(s)}, \quad v(0) = v_0, \quad |a_y| \le \bar{a}_y, \quad |a_t| \le \bar{a}_t, \quad \sqrt{a_t^2 + a_y^2} \le \bar{a}, \quad |j_y| \le \bar{j}_y$$

ここでは $L_{\text{in}} = 100$ m, $L = 110$ m, $L_{\text{out}} = 60$ m, $v_0 = 20$ m/s, $(\bar{a}_y, \bar{a}_t, \bar{a}) = (3.0, 2.0, 3.5)$ m/s², $\bar{j}_y = 0.8$ m/s³ とする.

表 16: 問題 7a の定式化 (NN は使わない)

項目	内容
学習器	なし (NN は使わない)
求める量	速度の 2 乗 $b = v^2$ の格子点の値 $b_1, \dots, b_n$ ($\Delta s = 1$ m, $n = 271$)
方程式	$a_y = b\,\kappa$, $a_t = b'/2$ (b について線形)
境界	$b_1 = v_0^2$
データ	なし
制約 (凸)	$\lvert a_y\rvert \le \bar{a}_y$, $\lvert a_t\rvert \le \bar{a}_t$, $\sqrt{a_t^2 + a_y^2} \le \bar{a}$ (二次錐)
制約 (非凸)	$\lvert j_y\rvert \le \bar{j}_y$ (線形化して解き直す: 逐次凸化)
目的	$T = \int b^{-1/2}\, ds$ を最小 (凸)

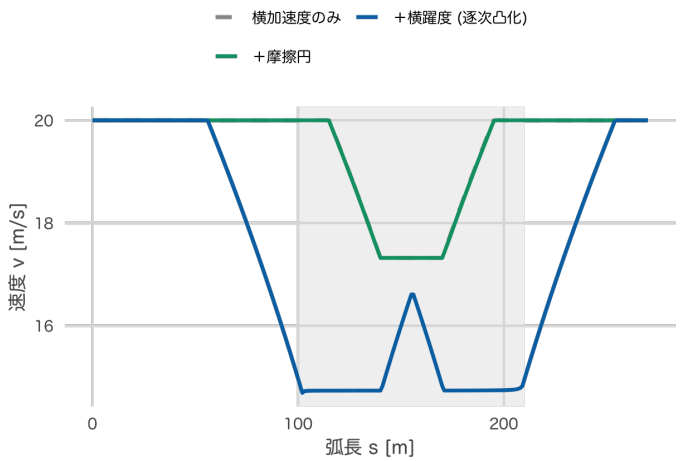
速度の 2 乗 $b(s) = v(s)^2$ を変数にとると, 横加速度 $a_y = b\kappa$ と接線加速度 $a_t = b'/2$ は b について線形, 摩擦円 $\sqrt{a_t^2 + (b\kappa)^2} \le \mu g$ は二次錐制約, 走行時間 $\int b^{-1/2} ds$ は凸になり, 最短時間の速度は二次錐計画 (SOCP) として大域最適に解ける (Lipp & Boyd, 2014; Verscheure et al., 2009). 横躍度 $j_y = v(b\kappa' + b'\kappa)$ は $v = b^{1/2}$ を含み非凸なので, 前回の解 v_k で $j_y \approx v_k(b\kappa' + b'\kappa)$ と線形化して凸問題を解き直す (逐次凸化). 振動を防ぐため更新に減衰を付け, 10 回反復した. 1 m 刻み 271 点, ソルバーは Clarabel (cvxpy 経由) である.

結果: 横加速度だけを課すと曲線中で 17.3 m/s まで減速し, 走行時間は 13.96 s. 摩擦円を加えても今回は同じだった. 横躍度を加えると曲率が変わるクロソイド区間でさらに減速が必要になり, 最低速度 14.7 m/s, 走行時間 16.02 s と 2 秒延びた. 計算時間は 1 回の SOCP が 20〜35 ms, 逐次凸化 11 回で計 281 ms である. 計算時間は点数・ソルバー・計算機に大きく依存する.

意味: 幾何が決まっていれば, 速度は古典的な凸最適化で大域最適に解け, 学習の出番はない. ただし幾何を固定すると, 制約を守るには減速するしかない.

4.8.2 問題 7b: 速度を保って走れる曲線

始点の位置・方位・曲率と出口の直線 (問題 4 の終点を通り方位 φ_f) が与えられたとき, 一定の速度 v のまま横加速度と横躍度の上限 ($|v^2\kappa| \le \bar{a}_y$, $|v^3\, d\kappa/ds| \le \bar{j}_y$) を守って走れ, 出口の直線に方位を合わせて曲率 0 で入る曲線と, その長さ L を求める. ここでは $v = 20$ m/s, $\varphi_f = 0.70$ rad とする. 終点は出口の直線上ならどこでもよい.



(a) 問題 4 の幾何 (網掛け) を走る最短時間の速度

制約	T [s]	j_y	ms
a_y	14.0	2.00	35
＋摩擦	14.0	2.00	23
＋ j_y	16.0	0.80	281

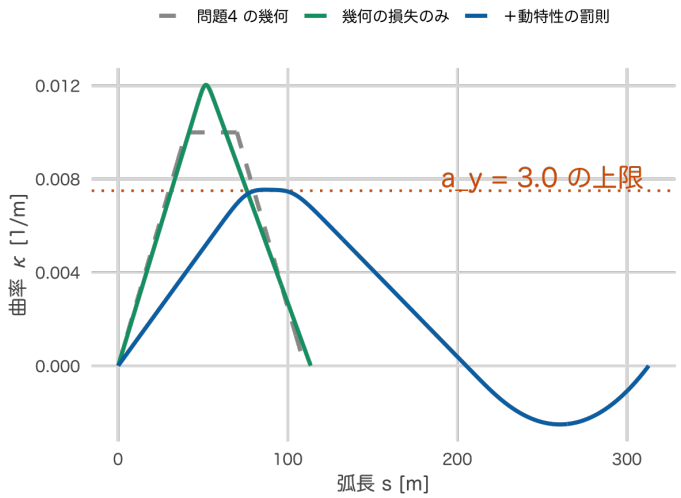
(b) 走行時間・横躍度の最大値・計算時間

図 14: 問題 7a の結果

表 17: 問題 7b の定式化

項目	内容
学習器	MLP f_θ: $t \mapsto j$．後ろに積分層と射影 (問題 4 と同じ)
学習する量	NN のパラメータ θ と物理パラメータ L (長さ)
方程式 $\mathcal{L}_r$	0 (積分層で構造的に満たす)
境界 $\mathcal{L}_b$	射影: $\kappa(1)=0$，$\varphi(1)=\varphi_f$，出口の直線までの距離 0
データ $\mathcal{L}_d$	なし
滑らかさ $\mathcal{L}_s$	使う
制約 $\mathcal{L}_c$	動特性 $g=(v^2\lvert\kappa\rvert/\bar{a}_y-1,\ v^3\lvert d\kappa/ds\rvert/\bar{j}_y-1)$
損失	$\mathcal{L}=\mathcal{L}_s+w_c\mathcal{L}_c+w_\ell L$，$(w_c,w_\ell)=(10^4,0.02)$

上限を超えた分だけ罰則がかかるように制約の項 $\mathcal{L}_c$ (ReLU の 2 乗) を使い，$w_\ell L$ で曲線を短くする圧力をかける．学習は 6,000 反復，L の初期値 170 m，学習率を最終的に 1/10 まで減衰させた．3,000 反復・減衰 1/20 では収束の途中で止まり $a_y=3.31$，$j_y=0.91$ だった．罰則の重みを 100 倍にしても結果は変わらず，Adam が勾配の大きさを正規化するため，重みより反復数と学習率の方が効いた．



(a) 同じ角 (40°) を 20 m/s のまま曲がる軌道の曲率

	L	a_y	j_y
問題 4	110	4.00	2.00
幾何	113	4.81	1.92
動特性	313	3.02	0.80

(b) 20 m/s で走ったときの長さと横加速度・横躍度の最大値

図 15: 問題 7b の結果

結果: 幾何の損失だけ ($w_c=0$) では $L=113$ m，最小半径 83 m で，$a_y=4.81$，$j_y=1.92$ と上限を大きく超えた．動特性の罰則を入れると $L=313$ m，最小半径 133 m で，$a_y=3.02$，$j_y=0.80$ と上限付近まで下がった．罰則法なので上限をわずかに超える (a_y で+0.6%，j_y で +0.3%)．

意味: 動特性を損失に入れると，速度を落とさずに走れる幾何が得られる．分離型は「減速して守る」，統合型は「大回りして守る」解で，どちらがよいかは用地や走行時間との兼ね合いによる．損失の重み w_ℓ がその選択

4. 走れる軌道には，凸最適化との分担か動特性の罰則が要る (問題 7a・7b).

表 18: 事例で現れた課題と主な対策

課題	事例での現れ方	主な対策
損失重みの不均衡	問題 3: w_d で外挿誤差が 0.2〜2.5 m と変動	勾配統計・NTK による自動重み付け
最適化の悪条件性	問題 2: Adam 2,700 回でも κ_1 が 84%	Adam → L-BFGS，二次法，無次元化
局所解	問題 3: $w_d = 1000$ で 3/5 試行が $A \approx 122$〜129 m	複数初期値，古典解からのウォームスタート
制約の強さ	問題 4: 罰則では終点 4 cm・曲率 79% のずれ	積分層・射影によるハード制約
計算コスト	問題 4: 1 件約 70 秒 → 問題 5: 0.7 ms	条件付き PINN，作用素学習
開曲線とフーリエ層	問題 6: 段差 1.5 rad の端で 0.67 rad の振動	トレンド除去・反射拡張，出力の選び方
動特性の保証	問題 7b: 罰則では上限を 0.3〜0.6% 超える	凸最適化との分担 (問題 7a)，制約の射影

5 古典的な数値解析との使い分け

4 節の事例は，PIML と古典的な数値解析 (数値積分，シューティング法，最小二乗，凸最適化) の得意・不得意を数値で比べる材料にもなる．

表 19: PIML と古典数値解析の使い分け

観点	古典数値解析	PIML	事例での根拠
1 つの問題を速く正確に解く	得意 (誤差評価と収束の理論がある)	遅く，精度も劣る	問題 1: 数値積分 0.3 ms・0.02 mm ／ PINN 4 s・6〜17 mm
未知数が少ない設計・同定	得意 (シューティング，最小二乗)	精度は同程度か劣り，遅い	問題 2: 0.1 s 未満 ／ 5 s．問題 3: 0.12〜0.98 m ／ 0.17〜1.72 m
未知が関数 (形が分からない)	解の形 (区間の構成) を決めてから解く	形を教えずに求まる (構造と組み合わせて)	問題 4: 構成を知れば 0.2 s ／ 構成なしで曲率誤差 3.4%
条件を変えて何度も解く	そのたびに解き直す	得意 (写像を学べば推論は 1 ms 未満)	問題 5: 1 件 25 s → 0.7 ms
観測と方程式を組み合わせる	モデルの形 (少数のパラメータ) が要る	得意 (同じ損失に足すだけ)	問題 3: データのみの 3 m 前後 → 物理を足して 0.2〜1.7 m
制約を厳密に守る	得意 (凸最適化なら大域最適)	罰則ではわずかに破る	問題 7a: SOCP で厳密 ／ 問題 7b: 上限を 0.3〜0.6% 超過

方程式も入力もすべて分かっている順問題では，古典的な数値積分が圧倒的に速く正確で，誤差評価や収束の理論も整っている．未知のパラメータが数個の設計・同定問題でも，シューティング法や非線形最小二乗が速く確実である．PIML の利点が出るのは，(1) 曲率分布のように未知が関数で，解の形を仮定できない場合，(2) 同じ問題を条件を変えて繰り返し解く場合，(3) 柔軟なモデルで観測と方程式を同時に扱う場合である．一方，制約の厳密な充足は古典的な最適化の方が確実で，罰則による PIML はわずかに制約を破る．

事例の後半で用いた積分層 (台形則)，射影 (Levenberg–Marquardt 法)，凸最適化 (二次錐計画) は，いずれも古典的な数値解析・最適化を NN の前後に組み込んだものである．同様の融合は文献にも多い．

表 20: 数値解析を取り込んだ手法の例

手法	取り入れた数値解析	何が改善するか
VPINN (Kharazmi et al., 2019)	有限要素法の弱形式	導関数の階数を下げ，求積が正確に
離散時間 PINN (Raissi et al., 2019)	陰的ルンゲ・クッタ法 (多段)	長い時間刻みで安定に進める
MGNO (Li et al., 2020)	高速多重極法の階層構造	長距離相互作用を $O(N)$ で
MANO (Colagrande et al., 2025)	多重極の考え方を注意機構に	注意機構を線形の計算量に
本稿の事例 (問題 4〜7b)	台形則・Levenberg–Marquardt 法・二次錐計画	運動学・終点条件・動特性を厳密に

VPINN は方程式にテスト関数を掛けて積分し部分積分で導関数を移す弱形式を損失に使い，NN に要求する導関数の階数を下げる．離散時間 PINN は時間方向を陰的ルンゲ・クッタ法で離散化し，NN が各段の値を出力する．MGNO は高速多重極法の階層的な分解を作用素学習のグラフ構造に取り入れ，MANO は多重極の考え方を注意機構に取り入れて大域的な相互作用を保ちつつ計算量を線形に抑える (後者は比較的新しい提案で評価は進行中)．

これらに共通する設計原理は，*確実に計算できる部分 (積分・射影・凸最適化) は古典解法に任せ，学習には関数の未知と繰り返しを任せる* ことである．すべてを NN の学習に任せるのではなく，学習に任せる部分と数値解析で確実に計算する部分を分けるのが，近年の PIML の大きな流れと言える．

6 課題と展望

事例で現れた課題 (4.9 節の表) を，研究の動向と合わせて整理する．

損失の重みと最適化: 複数の損失項の重みの選択は，PINN の実用上の最大の論点の一つである．勾配の統計による自動重み付け (Wang, Teng, et al., 2021)，NTK に基づく重み (Wang et al., 2022)，自己適応重み (McClenny & Braga-Neto, 2023) などが提案されているが，問題 3 で見たように重みは局所解への落ち込みとも関わり，自動化の決定版はない．損失の地形の悪条件性に対しては二次法の利用が標準になりつつあり (Rathore et al., 2024)，学習の手順 (正規化，学習率の減衰，因果性を考慮した重み付けなど) の実践的な指針も整理されている (Wang et al., 2023)．本稿の問題 4 のように，物理や条件を構造に移して損失の項そのものを減らすことは，重み付けの問題を設計で消す有効な選択肢である．

制約の保証: 罰則による制約は近似的にしか満たされない．安全性が要る応用では，境界条件を構造で満たす試行解 (Lagaris et al., 1998; Sukumar & Srivastava, 2022)，積分層や射影のような微分可能な層，凸最適化との分担など，制約を厳密に満たす仕組みとの組合せが必要になる．微分可能な射影や最適化層を学習中にどこまで使えるかは，学習の安定性との兼ね合いで問題ごとに判断が要る (問題 5 では推論時だけに置いた)．

スペクトルバイアスと多重スケール: 本稿の事例では解が滑らかでスペクトルバイアスは問題にならなかったが，衝撃波や境界層，多重スケールの現象では本質的な障害となる．入力の符号化 (Tancik et al., 2020; Wang, Wang, et al., 2021b)，周期活性化 (Sitzmann et al., 2020)，カリキュラム学習 (Krishnapriyan et al., 2021) などの対策があるが，汎用的な解決には至っていない．

求積と標本化: コロケーション点の取り方は損失の偏りに直結する．適応的サンプリングや，積分が入れ子になる損失での偏りの補正は，NeRF などの陰関数表現の研究とも共通する課題である．

作用素学習の汎化と境界: NO は学習した条件の分布の外では精度が保証されない．問題 6 のように，周期性を仮定するフーリエ層を開曲線や非周期の境界値問題に使うときは，何を層に通すかの設計が結果を左右する．チェビシェフ基底や領域の拡張など，非周期の問題への対応は実装上の重要な選択である．公平な比較のためのデータと評価の整備も進められている (Lu et al., 2022)．

フィジカル AI との統合: フィジカル AI の文脈では，PINN / PINO は方程式のソルバーにとどまらず，不完全な入力から，物理法則・幾何構造・アクチュエータの限界に適合した行動や軌道を実時間で生成する基盤として位置付けられる．本稿の問題 7a・7b はその小さな例である．VLA モデルの出力の物理的妥当性の保証，Sim-to-Real ギャップを縮めるためのパラメータ同定 (問題 3 の延長)，計算資源の限られた機体での高速な推論 (問題 5・6 の延長) が，今後の具体的な接点になる．

7 研究の歩み

2 節で見たように，物理を学習に組み込む考え方そのもの (正則化，制約，作用素の近似) は古い．それを支えた技術の時系列は，考え方の流れとは必ずしも一致しない．本節では，技術の年表で PIML の歩みを振り返る．

7.1 技術の年表

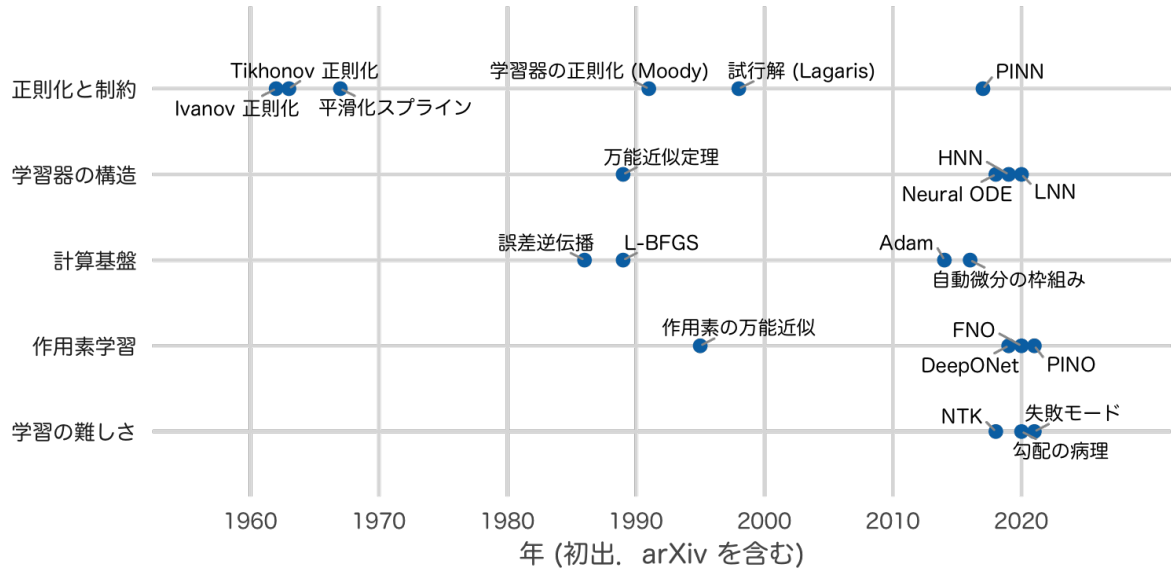


図 16: PIML を支える技術の年表 (年は初出．arXiv を含む目安)

年表の 5 つの系列は次のとおりである．

- **正則化と制約**: Ivanov と Tikhonov の正則化 (1962〜63) (Tikhonov & Arsenin, 1974/1977)，平滑化スプライン (1967) (Wahba, 1990)，学習器の微分を含む一般の正則化 (Moody, 1991)，境界条件を構造で満たす試行解 (Lagaris et al., 1998)，方程式の罰則を自動微分で計算する PINN (Raissi et al., 2019) (arXiv 2017)．2.3 節で見た罰則と制約の 2 つの書き方が，学習バイアスと帰納バイアスの源流である．
- **学習器の構造**: 関数の万能近似定理 (Cybenko, 1989; Hornik et al., 1989)，Neural ODE (R. T. Q. Chen et al., 2018)，HNN (Greydanus et al., 2019)，LNN (Cranmer et al., 2020)．
- **計算基盤**: 誤差逆伝播 (1986) (Goodfellow et al., 2016)，L-BFGS (D. C. Liu & Nocedal, 1989)，Adam (Kingma & Ba, 2015) (arXiv 2014)，自動微分を備えた深層学習の枠組み (TensorFlow 2015，PyTorch 2016〜17，JAX 2018)．微分を入れ子で数行に書けるようになったことが，PINN の実用化の鍵である (3.3 節)．
- **作用素学習**: 作用素の万能近似定理 (T. Chen & Chen, 1995)，DeepONet (Lu, Jin, et al., 2021) (arXiv 2019)，GNO (Anandkumar et al., 2020) と FNO (Li et al., 2021) (arXiv 2020)，PINO (Li et al., 2024) (arXiv 2021)．
- **学習の難しさ**: NTK (Jacot et al., 2018)，勾配の病理 (Wang, Teng, et al., 2021) (arXiv 2020)，失敗モードの整理 (Krishnapriyan et al., 2021)．

古い考え (正則化，作用素の近似) が，計算基盤の整った 2010 年代後半に一気に実用になった，というのが全体の流れである．

8 むすび

本稿では，物理を組み込む機械学習を，データの損失・正則化・仮説空間からなる古典的な学習の枠組みから見直し，物理を組み込む 3 つの場所 (観測・学習・帰納バイアス) と，それぞれを実現する技術として整理した．学習バイアスは物理残差を正則化項にする Tikhonov 型の方法で，自動微分と数値積分がそれを計算可能にした．帰納バイアスは学習器の後ろに固定の写像を置いて方程式や条件を構造で満たす Ivanov 型の方法で，試行

解，積分層と射影層，ハミルトン力学の構造がその例である．クロソイド曲線の設計を題材とした 8 つの問題では，問題を少しずつ難しくするたびに新しい技術 (設計変数の同時学習，データ項，構造によるハード制約，条件付き学習，作用素学習，凸最適化との分担，動特性の罰則) が必要になる様子を，統一した定式化と数値で示した．得られた教訓は，PIML を古典的な数値解析の代替としてではなく，それと分担して使うことの重要性である．確実に計算できる部分は古典解法に任せ，関数の未知と繰り返しを学習に任せる．この分担の設計こそが，フィジカル AI における物理の知識の活かし方の核心であると考える．

9 生成 AI の利用について

本稿と講演資料の作成では，2026 年 8〜10 月に生成 AI を補助に用いた．Claude (Anthropic) は，構成の検討，文章の下書きと推敲，数値実験と作図のコード，ブラウザデモの作成に用いた．Gemini (Google) は，文献の探索と要約，質疑応答による内容の検討など，内容の整理に用いた．数値実験の結果はすべてコードを実行して得たものである．引用した文献は書誌情報を確認した．内容についての責任は著者にある．

10 付録: 数値実験の設定

表 21: 数値実験の主な設定 (すべて CPU，float64)

問題	学習器	最適化	計算時間
1	MLP (中間層 3 層・各 32，tanh)	Adam 3,000 回 (学習率 3e-3) → L-BFGS 300 回	約 4 s (5 シード)
2	同上	Adam 3,000 回 → L-BFGS 300 回 (5 シード)	約 5 s / ケース
3	同上	Adam 4,000 回 → L-BFGS 300 回 (5 シード)	—
4	MLP (中間層 3 層・各 64，tanh，最終層 0 初期化)	Adam 4,000 回	約 70 s / 件
5	条件付き MLP	4,000 ステップ	学習 約 230 s，推論 0.7 ms / 件
6	FNO (幅 32，16 モード，4 層)	6,000 ステップ	学習 約 170 s
7a	なし (SOCP，271 点，Clarabel)	逐次凸化 10 回	281 ms
7b	MLP ＋ 積分層 ＋ 射影	Adam 6,000 回 (学習率を 1/10 まで減衰)	—

コロケーション点は問題 1〜3 で区間 $[0, 1]$ に等間隔 128 点．問題 4 以降の積分層は台形則の累積和で，射影の基底は低次のルジャンドル多項式 4 本，係数は Levenberg–Marquardt 法 (前の反復の係数から開始) で決めた．数値実験のコードは `code/` と `code/ladder/=`，結果の数値は `=data/` にあり，本稿の図表はそれを読み込んで作成している．問題 1〜3 は乱数シードを 5 通り変えた計算 (2026-10-13，Apple シリコンの CPU) で，表と図はシード 0 の結果を示した．ほかは 2026-10-05 の計算である．計算時間は機械によって数倍変わる．再実行して `data/` と比べる手順は `code/rerun_check.sh` にある．

参考文献

Anandkumar, A., Azizzadenesheli, K., Bhattacharya, K., Kovachki, N., Li, Z., Liu, B., & Stuart, A. (2020, February 26). *Neural Operator: Graph Kernel Network for Partial Differential Equations*. ICLR 2020 Workshop on Integration of Deep Neural Models and Differential Equations. https://openreview.net/forum?id=fg2ZFmXFO3

Barron, A. R. (1993). Universal approximation bounds for superpositions of a sigmoidal function. *IEEE Transactions on Information Theory*, *39*(3), 930–945. https://doi.org/10.1109/18.256500

Baydin, A. G., Pearlmutter, B. A., Radul, A. A., & Siskind, J. M. (2018). Automatic Differentiation in Machine Learning: A Survey. *Journal of Machine Learning Research*, *18*(153), 1–43. https://jmlr.org/papers/v18/17-468.html

Bertolazzi, E., & Frego, M. (2015). G1 fitting with clothoids. *Mathematical Methods in the Applied Sciences*, *38*(5), 881–897. https://doi.org/10.1002/mma.3114

Bishop, C. M. (1995). *Neural Networks for Pattern Recognition*. Clarendon Press (Oxford University Press).

Brunton, S. L., Proctor, J. L., & Kutz, J. N. (2016). Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, *113*(15), 3932–3937. https://doi.org/10.1073/pnas.1517384113

Chen, R. T. Q., Rubanova, Y., Bettencourt, J., & Duvenaud, D. K. (2018). Neural Ordinary Differential Equations. *Advances in Neural Information Processing Systems*, *31*. https://proceedings.neurips.cc/paper_files/paper/2018/hash/69386f6bb1dfed68692a24c8686939b9-Abstract.html

Chen, T., & Chen, H. (1995). Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems. *IEEE Transactions on Neural Networks*, *6*(4), 911–917. https://doi.org/10.1109/72.392253

Colagrande, A., Caillon, P., Feillet, E., & Allauzen, A. (2025). Linear Attention with Global Context: A Multipole Attention Mechanism for Vision and Physics. *2025 IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, 3130–3139. https://doi.org/10.1109/ICCVW69036.2025.00326

Cranmer, M., Greydanus, S., Hoyer, S., Battaglia, P., Spergel, D., & Ho, S. (2020). Lagrangian Neural Networks. *ICLR 2020 Workshop on Integration of Deep Neural Models and Differential Equations*. https://openreview.net/forum?id=iE8tFa4Nq

Cuomo, S., Di Cola, V. S., Giampaolo, F., Rozza, G., Raissi, M., & Piccialli, F. (2022). Scientific Machine Learning Through Physics–Informed Neural Networks: Where we are and What's Next. *Journal of Scientific Computing*, *92*(3), 88. https://doi.org/10.1007/s10915-022-01939-z

Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, *2*(4), 303–314. https://doi.org/10.1007/BF02551274

E, W., & Yu, B. (2018). The Deep Ritz Method: A Deep Learning-Based Numerical Algorithm for Solving Variational Problems. *Communications in Mathematics and Statistics*, *6*(1), 1–12. https://doi.org/10.1007/s40304-018-0127-z

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.

Greydanus, S., Dzamba, M., & Yosinski, J. (2019). Hamiltonian Neural Networks. *Advances in Neural Information Processing Systems*, *32*. https://proceedings.neurips.cc/paper_files/paper/2019/hash/26cd8ecadce0d4efd6cc8a8725cbd1f8-Abstract.html

Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, *2*(5), 359–366. https://doi.org/10.1016/0893-6080(89)90020-8

Jacot, A., Gabriel, F., & Hongler, C. (2018). Neural Tangent Kernel: Convergence and Generalization in Neural Networks. *Advances in Neural Information Processing Systems*, *31*. https://proceedings.neurips.cc/paper_files/paper/2018/hash/5a4be1fa34e62bb8a6ec6b91d2462f5a-Abstract.html

Karniadakis, G. E., Kevrekidis, I. G., Lu, L., Perdikaris, P., Wang, S., & Yang, L. (2021). Physics-informed machine learning. *Nature Reviews Physics*, *3*(6), 422–440. https://doi.org/10.1038/s42254-021-00314-5

Kharazmi, E., Zhang, Z., & Karniadakis, G. E. (2019, November 27). *Variational Physics-Informed Neural Networks For Solving Partial Differential Equations*. https://doi.org/10.48550/arXiv.1912.00873

Kim, S.-J., Koh, K., Boyd, S., & Gorinevsky, D. (2009). ℓ_1 Trend Filtering. *SIAM Review*, *51*(2), 339–360. https://doi.org/10.1137/070690274

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In Y. Bengio & Y. LeCun (Eds.), *3rd international conference on learning representations, ICLR 2015, san diego, CA, USA, may 7-9, 2015, conference track proceedings*. https://doi.org/10.48550/ARXIV.1412.6980

Kovachki, N., Lanthaler, S., & Mishra, S. (2021). On Universal Approximation and Error Bounds for Fourier Neural Operators. *Journal of Machine Learning Research*, *22*(290), 1–76. https://jmlr.org/papers/v22/21-0806.html

Kovachki, N., Li, Z., Liu, B., Azizzadenesheli, K., Bhattacharya, K., Stuart, A., & Anandkumar, A. (2023). Neural Operator: Learning Maps Between Function Spaces With Applications to PDEs. *Journal of Machine Learning Research*, *24*(89), 1–97. http://jmlr.org/papers/v24/21-1524.html

Krishnapriyan, A., Gholami, A., Zhe, S., Kirby, R., & Mahoney, M. (2021). Characterizing possible failure modes in physics-informed neural networks. *Advances in Neural Information Processing Systems*, *34*, 26548–26560. https://proceedings.neurips.cc/paper_files/paper/2021/hash/df438e5206f31600e6ae4af72f2725f1-Abstract.html

Lagaris, I. E., Likas, A., & Fotiadis, D. I. (1998). Artificial neural networks for solving ordinary and partial differential equations. *IEEE Transactions on Neural Networks*, *9*(5), 987–1000. https://doi.org/10.1109/72.712178

Li, Z., Kovachki, N. B., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A. M., & Anandkumar, A. (2021). Fourier neural operator for parametric partial differential equations. *9th International*

Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021. https://openreview.net/forum?id=c8P9NQVtmnO

Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Stuart, A., Bhattacharya, K., & Anandkumar, A. (2020). Multipole Graph Neural Operator for Parametric Partial Differential Equations. *Advances in Neural Information Processing Systems*, *33*, 6755–6766. https://proceedings.neurips.cc/paper_files/paper/2020/hash/4b21cf96d4cf612f239a6c322b10c8fe-Abstract.html

Li, Z., Zheng, H., Kovachki, N., Jin, D., Chen, H., Liu, B., Azizzadenesheli, K., & Anandkumar, A. (2024). Physics-Informed Neural Operator for Learning Partial Differential Equations. *ACM/JMS Journal of Data Science*, *1*(3), 9:1–9:27. https://doi.org/10.1145/3648506

Lipp, T., & Boyd, S. (2014). Minimum-time speed optimisation over a fixed path. *International Journal of Control*, *87*(6), 1297–1311. https://doi.org/10.1080/00207179.2013.875224

Liu, D. C., & Nocedal, J. (1989). On the limited memory BFGS method for large scale optimization. *Mathematical Programming*, *45*(1), 503–528. https://doi.org/10.1007/BF01589116

Liu, Z., Wang, Y., Vaidya, S., Ruehle, F., Halverson, J., Soljacic, M., Hou, T., & Tegmark, M. (2025). KAN: Kolmogorov–Arnold Networks. *International Conference on Learning Representations*, *2025*, 70367–70413. https://proceedings.iclr.cc/paper_files/paper/2025/hash/afaed89642ea100935e39d39a4da602c-Abstract-Conference.html

Lu, L., Jin, P., Pang, G., Zhang, Z., & Karniadakis, G. E. (2021). Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, *3*(3), 218–229. https://doi.org/10.1038/s42256-021-00302-5

Lu, L., Meng, X., Cai, S., Mao, Z., Goswami, S., Zhang, Z., & Karniadakis, G. E. (2022). A comprehensive and fair comparison of two neural operators (with practical extensions) based on FAIR data. *Computer Methods in Applied Mechanics and Engineering*, *393*, 114778. https://doi.org/10.1016/j.cma.2022.114778

Lu, L., Meng, X., Mao, Z., & Karniadakis, G. E. (2021). DeepXDE: A Deep Learning Library for Solving Differential Equations. *SIAM Review*, *63*(1), 208–228. https://doi.org/10.1137/19M1274067

McClenny, L. D., & Braga-Neto, U. M. (2023). Self-adaptive physics-informed neural networks. *Journal of Computational Physics*, *474*, 111722. https://doi.org/10.1016/j.jcp.2022.111722

Mildenhall, B., Srinivasan, P. P., Tancik, M., Barron, J. T., Ramamoorthi, R., & Ng, R. (2021). NeRF: Representing scenes as neural radiance fields for view synthesis. *Commun. ACM*, *65*(1), 99–106. https://doi.org/10.1145/3503250

Miriyev, A., & Kovač, M. (2020). Skills for physical artificial intelligence. *Nature Machine Intelligence*, *2*(11), 658–660. https://doi.org/10.1038/s42256-020-00258-y

Mitchell, T. M. (1997). *Machine learning*. McGraw-Hill. https://books.google.co.jp/books?id=EoYBngEACAAJ

Moody, J. (1991). The Effective Number of Parameters: An Analysis of Generalization and Regularization in Nonlinear Learning Systems. *Advances in Neural Information Processing Systems*, *4*. https://proceedings.neurips.cc/paper/1991/hash/d64a340bcb633f536d56e51874281454-Abstract.html

Müller, T., Evans, A., Schied, C., & Keller, A. (2022). Instant neural graphics primitives with a multiresolution hash encoding. *ACM Transactions on Graphics*, *41*(4), 102:1–102:15. https://doi.org/10.1145/3528223.3530127

Négiar, G., Mahoney, M. W., & Krishnapriyan, A. S. (2023, May). Learning differentiable solvers for systems with hard constraints. *The Eleventh International Conference on Learning Representations*. https://openreview.net/forum?id=vdv6CmGksr0

Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, *378*, 686–707. https://doi.org/10.1016/j.jcp.2018.10.045

Rathore, P., Lei, W., Frangella, Z., Lu, L., & Udell, M. (2024). Challenges in Training PINNs: A Loss Landscape Perspective. *Proceedings of the 41st International Conference on Machine Learning*, 42159–42191. https://proceedings.mlr.press/v235/rathore24a.html

Sirignano, J., & Spiliopoulos, K. (2018). DGM: A deep learning algorithm for solving partial differential equations. *Journal of Computational Physics*, *375*, 1339–1364. https://doi.org/10.1016/j.jcp.2018.08.029

Sitzmann, V., Martel, J., Bergman, A., Lindell, D., & Wetzstein, G. (2020). Implicit Neural Representations with Periodic Activation Functions. *Advances in Neural Information Processing Systems*, *33*, 7462–7473. https://proceedings.neurips.cc/paper_files/paper/2020/hash/53c04118df112c13a8c34b38343b9c10-Abstract.html

Sukumar, N., & Srivastava, A. (2022). Exact imposition of boundary conditions with distance functions in physics-informed deep neural networks. *Computer Methods in Applied Mechanics and Engineering*, *389*, 114333. https://doi.org/10.1016/j.cma.2021.114333

Tancik, M., Srinivasan, P., Mildenhall, B., Fridovich-Keil, S., Raghavan, N., Singhal, U., Ramamoorthi,

R., Barron, J., & Ng, R. (2020). Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains. *Advances in Neural Information Processing Systems*, *33*, 7537–7547. https://proceedings.neurips.cc/paper_files/paper/2020/hash/55053683268957697aa39fba6f231c68-Abstract.html

Thakur, A., Kaipa, K., Banerjee, A. G., Cappelleri, D. J., Krovi, V. N., & Gupta, S. (2025). Physical Artificial Intelligence for Powering the Next Revolution in Robotics. *Journal of Computing and Information Science in Engineering*, *25*(12), 120809. https://doi.org/10.1115/1.4070122

Tikhonov, A. N., & Arsenin, V. Y. (1977). *Solutions of Ill-posed Problems* (F. John, Trans.). Winston. https://books.google.com?id=ECrvAAAAMAAJ (Original work published as **Методы решения некорректных задач**, 1974)

Verscheure, D., Demeulenaere, B., Swevers, J., De Schutter, J., & Diehl, M. (2009). Time-Optimal Path Tracking for Robots: A Convex Optimization Approach. *IEEE Transactions on Automatic Control*, *54*(10), 2318–2327. https://doi.org/10.1109/TAC.2009.2028959

Wahba, G. (1990). *Spline models for observational data*. Society for Industrial and Applied Mathematics (SIAM). https://doi.org/10.1137/1.9781611970128

Wang, S., Sankaran, S., Wang, H., & Perdikaris, P. (2023, August 16). *An Expert's Guide to Training Physics-informed Neural Networks*. https://doi.org/10.48550/arXiv.2308.08468

Wang, S., Teng, Y., & Perdikaris, P. (2021). Understanding and Mitigating Gradient Flow Pathologies in Physics-Informed Neural Networks. *SIAM Journal on Scientific Computing*, *43*(5), A3055–A3081. https://doi.org/10.1137/20M1318043

Wang, S., Wang, H., & Perdikaris, P. (2021a). Learning the solution operator of parametric partial differential equations with physics-informed DeepONets. *Science Advances*, *7*(40), eabi8605. https://doi.org/10.1126/sciadv.abi8605

Wang, S., Wang, H., & Perdikaris, P. (2021b). On the eigenvector bias of Fourier feature networks: From regression to solving multi-scale PDEs with physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, *384*, 113938. https://doi.org/10.1016/j.cma.2021.113938

Wang, S., Yu, X., & Perdikaris, P. (2022). When and why PINNs fail to train: A neural tangent kernel perspective. *Journal of Computational Physics*, *449*, 110768. https://doi.org/10.1016/j.jcp.2021.110768

Willard, J., Jia, X., Xu, S., Steinbach, M., & Kumar, V. (2022). Integrating Scientific Knowledge with Machine Learning for Engineering and Environmental Systems. *ACM Computing Surveys*, *55*(4), 66:1–66:37. https://doi.org/10.1145/3514228